Prompt Engineering Guide

A hands-on guide to writing effective prompts, building reusable templates, and automating client work with AI coding tools.

GitHub Copilot • Windsurf • Claude Code

8 Sections • 36 Exercises • ~5–7 Hours

Core concepts are tool-agnostic — learn once, apply anywhere.

Curated by Gustavo Andres

Created with the help of AI • Free to use for learning purposes only

About This Guide

How This Was Made

This guide was curated by **Gustavo Andres** and created collaboratively with the help of AI (Anthropic's Claude). The content — explanations, exercises, templates, diagrams, and this PDF itself — was drafted and refined through iterative prompting, exactly the same process the guide teaches. Human review, editing, and curation shaped the final material for accuracy and practical usefulness.

Treating this guide as a worked example is fair game: every prompting technique demonstrated here was used at some point to produce the guide itself.

License & Usage

Free to use for learning purposes only.

You are welcome to:

- Read, study, and practice with this guide personally or as a team.
- Share the PDF internally with colleagues for educational use.
- Adapt the templates and meta-prompts for your own work.
- Reference or cite portions of the guide with attribution.

Please do not:

- Resell, relicense, or distribute commercially without permission.
- Claim authorship or remove attribution.
- Use the content to train commercial AI models.

Disclaimer & Limitation of Liability

This guide is provided **"as-is"**, for educational and informational purposes only, without any warranties of any kind, express or implied, including but not limited to warranties of merchantability, fitness for a particular purpose, accuracy, or non-infringement.

Gustavo Andres (the curator) and any AI systems used in producing this material shall not be held liable for any direct, indirect, incidental, consequential, or other damages, losses, or issues arising from the use, misuse, or inability to use this guide. This includes, without limitation: incorrect or suboptimal

AI output, bugs introduced into code, production incidents, security vulnerabilities, data loss, contractual or client-related consequences, or any other side effects that may result from applying the techniques, templates, prompts, or examples described herein.

AI tools evolve quickly — feature names, file paths, and tool behaviors may change. You are solely responsible for verifying any technique or recommendation against your tool's current documentation, applying appropriate code review and testing, and ensuring that AI-generated output meets your project's correctness, security, and compliance requirements before using it in client or production work.

By using this guide, you acknowledge and accept that you do so entirely at your own risk.

Feedback

Found something that could be clearer or more accurate? Contributions and suggestions that improve the guide for the whole team are welcomed.

Prompt Engineering Guide — curated by Gustavo Andres.
Built with AI, shared for learning.

Table of Contents

About This Guide 2

Table of Contents 4

Introduction 9

- **Prompt Engineering Guide** . . . 10
  - How AI Prompting Actually Works (the 60-Second Mental Model) . . . 10
  - Who This Is For . . . 11
  - What's in This Guide . . . 11
  - Getting Started . . . 11
  - Learning Path . . . 11
  - How Concepts Map Across Tools . . . 12
  - Setting Up Your Team's Prompt Library . . . 13
  - Quick Wins . . . 13

Curriculum at a Glance 14

- **Curriculum at a Glance** . . . 15
  - What You'll Learn . . . 15
  - How the Guide Is Organized . . . 15
  - The Modules in 30 Seconds Each . . . 16
  - Two Paths to a Meta-Prompt . . . 17
  - Quick-Start Cheat Sheet . . . 17

Hands-On Workbook 18

- **Prompt Engineering — Complete Hands-On Workbook** . . . 19
  - How This Workbook Is Organized . . . 19

Module 1 — Prompt Foundations . . . 20

Exercise 1 — Goal-First Writing . . . 21

Exercise 2 — Providing Context Effectively . . . 22

Exercise 3 — Setting Constraints That Shape the Output . . . 23

Exercise 4 — Few-Shot Prompting (Teaching by Example) . . . 24

Exercise 5 — Chain-of-Thought Prompting . . . 25

Module 2 — Building Reusable Prompt Templates . . . 27

Exercise 6 — Build a Bug Fix Template . . . 27

Exercise 7 — Build a Code Generation Template . . . 28

Exercise 8 — Build a Code Review Template . . . 29

Exercise 9 — Build a Documentation Template . . . 30

Exercise 10 — Have AI Generate a Template for Any Task . . . 31

Module 3 — Build Your Own Meta-Prompt . . . 32

Exercise 11 — Define the Role and Purpose . . . 33

Exercise 12 — Create the Clarifying-Questions Protocol . . . 34

Exercise 13 — Add the Confirmation Step . . . 35

Exercise 14 — Define the Output Prompt Template . . . 36

Exercise 15 — Add the Review & Refine Loop . . . 37

Exercise 16 — Add Guardrails . . . 39

Exercise 17 — Add an Example Conversation (Optional but Recommended) . . . 39

Exercise 18 — Test Your Meta-Prompt . . . 40

Exercise 19 — Customize for Your Team . . . 41

Exercise 20 — Share and Version Your Meta-Prompt . . . 42

Module 4 — Advanced Prompting Strategies . . . 43

Exercise 21 — Iterative Refinement (Treat It Like a Draft) . . . 43

Exercise 22 — Multi-Persona Prompting . . . 44

Exercise 23 — Constraint-Based Generation . . . 45

Module 5 — GitHub Copilot . . . 47

Exercise 24 — Context References: #file, #selection, and Workspace Awareness . . . 47

Exercise 25 — Slash Commands: Built-In Prompt Shortcuts . . . 48

Exercise 26 — Custom Instructions: .github/copilot-instructions.md . . . 49

Module 6 — Windsurf (Cascade) . . . 51

Exercise 27 — Codebase-Aware Prompting with Cascade . . . 51

Exercise 28 — Windsurf Rules and Workflows . . . 52

Module 7 — Claude Code . . . 55

Exercise 29 — CLAUDE.md: Project Instructions . . . 55

Exercise 30 — Skills: Reusable Slash Commands . . . 57

Exercise 31 — Hooks and MCP Servers . . . 59

Module 8 — Automating Client Work . . . 61

Exercise 32 — Identify Automatable Tasks . . . 62

Exercise 33 — Build a Client-Specific Prompt Template . . . 62

Exercise 34 — Test and Validate Your Client Prompt . . . 63

Module 9 — Putting It All Together . . . 65

Exercise 35 — End-to-End Challenge . . . 65

Exercise 36 — Build a Team Prompt Library . . . 66

Quick Reference — When to Use What . . . 66

What's Next . . . 67

Build Your Own Meta-Prompt 68

Build Your Own Meta-Prompt with AI . . . 69

How This Section Relates to Module 3 . . . 69

The Big Idea — Three Layers . . . 69

How This Guide Works . . . 70

Step 1 — Define Your Target Use Case . . . 71

Step 2 — Ask AI to Draft Your Meta-Prompt . . . 72

Step 3 — Stress-Test Your Meta-Prompt . . . 73

Step 4 — Ask AI to Diagnose and Fix Problems . . . 74

Step 5 — Add Your Team's Context . . . 75

Step 6 — Operationalize It . . . 76

You're Done — What You Now Have . . . 78

The 3-Layer Pattern in Action (End-to-End Example) . . . 78
Tips for Long-Term Success . . . 79
How This Fits the Bigger Picture . . . 79

Quick Reference **79**

Prompt Engineering — Quick Reference . . . 81

The 6-Step Prompt Formula . . . 81
Prompt Patterns . . . 81
Tool-Specific Features . . . 82
Reusable Template Structure . . . 84
Common Templates . . . 84
Troubleshooting . . . 86
Iterative Refinement Phrases . . . 86
File Organization . . . 86

Meta-Prompt File **87**

Meta-Prompt: Generate High-Quality Prompts . . . 88

SYSTEM INSTRUCTIONS . . . 88
GUARDRAILS . . . 90
EXAMPLE CONVERSATION FLOW . . . 90

Practice Scenarios **91**

Practice Scenarios . . . 92

How to Use This Section . . . 92
Scenario 1 — Slow PostgreSQL Query . . . 92
Scenario 2 — React Component Re-Renders Excessively . . . 92
Scenario 3 — NullPointerException in Spring Boot Service . . . 93
Scenario 4 — Refactor: Extract Duplicated Validation Logic . . . 93
Scenario 5 — Generate Unit Tests for a Service Class . . . 93
Scenario 6 — Write API Documentation from Existing Code . . . 93

Scenario 7 — Architecture Decision: Monorepo vs. Multi-Repo 94
Scenario 8 — Database Migration: Add NOT NULL Column to Large Table 94
Scenario 9 — Code Review: Security Audit of Authentication Endpoint 94
Scenario 10 — Bug: Intermittent Test Failure in CI 95
Tips for Using These Scenarios 95

Glossary **95**

Glossary 96

Core Prompting Concepts 96
Prompting Techniques 96
GitHub Copilot Terms 97
Windsurf Terms 97
Claude Code Terms 98
Generic AI / LLM Terms 98

SECTION 1

Introduction

Prompt Engineering Guide

A hands-on guide that teaches our team how to write effective prompts, build reusable prompt templates, and automate client work using AI coding tools.

Core concepts are tool-agnostic — learn once, apply to any tool. Tool-specific modules cover GitHub Copilot, Windsurf, and Claude Code separately.

How AI Prompting Actually Works (the 60-Second Mental Model)

Before any techniques, here's the underlying concept everything in this guide builds on.

A **prompt** is just an instruction you give to an AI. Think of it like delegating a task to a brand-new colleague who has read everything ever written but knows nothing about *your* project, *your* codebase, *your* team's standards, or what "done" looks like *for you*. The AI is fast, knowledgeable, and willing — but it has zero context until you give it some.

That single insight shapes every technique in this guide:

What you provide	Why it matters
A clear goal	Tells the colleague what "done" looks like — the most common reason AI output disappoints is that the goal was vague
Relevant context	What project, what stack, what's been tried — without this the AI guesses (poorly)
Constraints	Format, length, audience, what to avoid — anchors the output to your actual needs
Examples (sometimes)	"Show me what good looks like" beats "explain what good looks like" every time
Permission to ask back	When in doubt, the AI should question you — silent guesses produce wrong answers

Three concepts you'll encounter often:

- **Prompt** — a single instruction you type into an AI chat
- **Prompt template** — a fill-in-the-blank prompt your team reuses (so the same task gets handled the same way every time)
- **Meta-prompt** — a prompt that *generates other prompts* — you build it once and it acts as a "prompt engineer" interviewing you to produce tailored prompts on demand

Every module in this guide is a deeper dive into one of these. The skills compound: a good prompt becomes a good template, and a good template becomes part of a good meta-prompt. Once you have all three, you stop "writing prompts" and start *teaching AI how to write prompts for you* — that's the compound-interest skill.

Who This Is For

Developers on our team who want to:

- Get better results from AI coding tools
- Create reusable prompt templates for common tasks (bug fixes, code gen, reviews, docs)
- Build client-specific prompts that any team member can use to automate repetitive work

No prior prompt engineering experience required.

What's in This Guide

This guide is organized into the following sections (see the Table of Contents to jump to any of them):

Section	What It Covers	When to Use It
Hands-On Workbook	35 exercises across 9 modules	The main learning path (~3.5 hrs)
Topic Outline	Full curriculum at a glance	Skim first to see the lay of the land
Quick Reference	Patterns, templates, tool features in a few pages	Flip back to it as a cheat sheet
Meta-Prompt File	A ready-to-use meta-prompt	Copy-paste into any AI chat to generate prompts on demand
Build Your Own Meta-Prompt	Fast-path guide (~45–60 min)	Build a personalized meta-prompt with AI's help

Getting Started

1. Skim the **Topic Outline** section first to see the full curriculum at a glance
2. Work through the **Hands-On Workbook** section at your own pace — each module is self-contained
3. Treat the **Quick Reference** section as a cheat sheet — flip back to it as needed while practicing
4. **For tool-specific modules (5–7)**: do the one(s) that match the tool(s) you have access to, skip the rest

Prerequisites

- At least one AI coding tool: **GitHub Copilot Chat**, **Windsurf**, or **Claude Code**
- A project repo you can use for practice exercises

Learning Path

```
CORE CONCEPTS (any tool) ──────────────────────────────────────────
│
│  Module 1 — Prompt Foundations          (Ex. 1–5)    ~20 min
│    Write clear, structured prompts with goals, context, constraints
│
│  Module 2 — Reusable Templates         (Ex. 6–10)   ~25 min
│    Build fill-in-the-blank templates for bug fixes, code gen, reviews, docs
│
│  Module 3 — Build Your Own Meta-Prompt (Ex. 11–20)  ~35 min
│    Create a "prompt that generates prompts"
│
│  Module 4 — Advanced Strategies        (Ex. 21–23)  ~20 min
│    Iterative refinement, multi-persona analysis, constraint-based generation
│
├─ TOOL-SPECIFIC (do the modules for tools you have access to) ────
│
│  Module 5 — GitHub Copilot             (Ex. 24–26)  ~20 min
│    #file, #selection, slash commands, .github/copilot-instructions.md
│    (workspace context is now implicit; was @workspace in older versions)
│
│  Module 6 — Windsurf                   (Ex. 27–28)  ~15 min
│    ⚠  Skip if you don't have Windsurf yet
│    @codebase, Cascade, .windsurf/rules/, .windsurf/workflows/
│
│  Module 7 — Claude Code                (Ex. 29–31)  ~15 min
│    ⚠  Skip if you don't have Claude Code yet
│    CLAUDE.md, .claude/skills/, hooks, MCP servers
│
├─ APPLYING IT ────────────────────────────────────────────────────
│
│  Module 8 — Automating Client Work     (Ex. 32–34)  ~20 min
│    Identify automatable tasks, build client templates, validate them
│
│  Module 9 — Putting It All Together    (Ex. 35)     ~15 min
│    End-to-end challenge and team prompt library
```

How Concepts Map Across Tools

The same ideas have different names in each tool:

Concept	GitHub Copilot	Windsurf	Claude Code
Project instructions	`.github/copilot-instructions.md`	`.windsurf/rules/*.md`	`CLAUDE.md`
Reusable prompts	`.github/prompts/*.prompt.md`	`.windsurf/workflows/*.md`	`.claude/skills/*.md`
Trigger saved prompt	`#<prompt-name>`	`/workflow-name`	`/skill-name`
File context	`#file:path`, `#selection`	`@file`	Automatic

Concept	GitHub Copilot	Windsurf	Claude Code
Codebase context	Implicit (was `@workspace`)	`@codebase`	Automatic
Automation hooks	—	—	Hooks (pre/post scripts)

Setting Up Your Team's Prompt Library

After completing the workbook, your repo should have:

```
your-repo/
├── .github/                          ← GitHub Copilot
│   ├── copilot-instructions.md       ←   Project instructions
│   └── prompts/*.prompt.md           ←   Reusable prompt files
├── .windsurf/                        ← Windsurf (when available)
│   ├── rules/*.md                    ←   Project instructions
│   └── workflows/*.md                ←   Reusable workflows
├── CLAUDE.md                         ← Claude Code (when available)
├── .claude/
│   └── skills/*.md                   ←   Reusable skills
├── prompts/                          ← Shared (any tool)
│   ├── meta-prompt.md
│   ├── templates/
│   │   ├── bug-fix.md
│   │   ├── code-gen.md
│   │   ├── code-review.md
│   │   └── documentation.md
│   └── client-[project]/
│       └── [task-specific templates]
```

The `prompts/` folder works with any tool — just paste template contents into chat. The tool-specific folders enable each tool's native features.

Quick Wins

If you only have 10 minutes, do these:

1. **Set up project instructions** for your current project (Exercise 26 for Copilot, Exercise 28 for Windsurf, Exercise 29 for Claude Code, all in the Hands-On Workbook section) — every conversation will automatically follow your team's standards
2. **Learn file references** (Exercise 24 in the Hands-On Workbook section) — stop pasting code, start referencing it
3. **Skim the Quick Reference section** — the 6-step formula and template structures

SECTION 2

Curriculum at a Glance

Curriculum at a Glance

*A 60-second overview of what's in this guide and why each section matters. The actual teaching happens in the **Hands-On Workbook** section — this page is just the map.*

What You'll Learn

By the end of this guide, you'll be able to:

1. **Write effective AI prompts** that produce useful results on the first try (instead of having to re-prompt repeatedly)
2. **Build reusable prompt templates** your team can fill in to get consistent output for common tasks
3. **Construct meta-prompts** — prompts that teach AI to generate prompts for you on demand
4. **Use your AI tool's power features** (GitHub Copilot, Windsurf, or Claude Code) to give the AI rich context without manual copy-paste
5. **Automate parts of client work** that are repetitive enough to benefit from a templated AI workflow

---

How the Guide Is Organized

#	Section	Purpose	When to Read
1	**Introduction**	Goal, audience, learning paths	Right now
2	**Curriculum at a Glance** *(this page)*	What's covered and why	Right now
3	**Hands-On Workbook**	36 exercises across 9 modules — the main learning	The bulk of your time
4	**Build Your Own Meta-Prompt**	A focused 6-step path to build a meta-prompt for one specific use case	After Module 3, or as a faster alternative
5	**Quick Reference**	Patterns, templates, and tool features condensed	Cheat sheet — flip back to it constantly
6	**Meta-Prompt File**	A polished, ready-to-use meta-prompt	Compare against your own; copy-paste when you don't want to build from scratch
7	**Practice Scenarios**	10 realistic tasks if you don't have a current real one	When an exercise says "use a real task" and you don't have one

#	Section	Purpose	When to Read
8	**Glossary**	One-line definitions of every term	When you forget what "few-shot" or "MCP" means

---

The Modules in 30 Seconds Each

Modules 1–4 — Tool-Agnostic Core

The foundational skills. Work in any AI tool. **Do these regardless of which AI assistant you use.**

- **Module 1 — Prompt Foundations** *(Exercises 1–5)*

Goal-first writing, context, constraints, examples (few-shot), and chain-of-thought reasoning. The five techniques every good prompt uses.

- **Module 2 — Reusable Templates** *(Exercises 6–10)*

Build fill-in-the-blank templates for the four tasks teams do most: bug fixes, code generation, code review, and documentation.

- **Module 3 — Build Your Own Meta-Prompt** *(Exercises 11–20)*

Construct a "prompt that generates prompts." Build it section by section. After Module 3, you have a generic meta-prompt anyone on the team can use.

- **Module 4 — Advanced Strategies** *(Exercises 21–23)*

Iterative refinement, multi-persona analysis, and constraint-based generation. Power tools for complex or trade-off-heavy tasks.

Modules 5–7 — Tool-Specific

Pick the module(s) for the tool(s) you have access to. Skip the rest.

- **Module 5 — GitHub Copilot** *(Exercises 24–26)*

`#file`, `#selection`, slash commands (`/explain`, `/fix`, `/tests`, `/doc`), and `.github/copilot-instructions.md` for team-wide standards.

- **Module 6 — Windsurf (Cascade)** *(Exercises 27–28)*

Codebase-aware prompting with `@codebase` and `@file`, plus `.windsurf/rules/` and `.windsurf/workflows/` for reusable team patterns.

- **Module 7 — Claude Code** *(Exercises 29–31)*

`CLAUDE.md` for project context, `.claude/skills/` for reusable slash commands, hooks for automation, and MCP servers for external integrations.

Modules 8–9 — Apply It to Real Work

- **Module 8 — Automating Client Work** *(Exercises 32–34)*

Identify which client tasks are good automation candidates, build a client-specific prompt template, and validate it against multiple real inputs.

- **Module 9 — Putting It All Together** *(Exercises 35–36)*

An end-to-end challenge using a real task, plus building a shared team prompt library so the work compounds across the team.

Two Paths to a Meta-Prompt

This guide gives you two ways to build a meta-prompt — pick one or do both:

Path	Where	When to Use
Generic, learn-by-doing	Module 3 of the Workbook	You want to understand every part of a meta-prompt; you'll use it for many different tasks
Specific, use-case-focused	Section 4 (Build Your Own Meta-Prompt)	You have one recurring task in mind; you want to build something tailored fast

If you have time for one: do **Module 3** first (you learn the parts), then **Section 4** (you apply that knowledge to a real recurring task).

If you're in a hurry: jump straight to **Section 4**.

Quick-Start Cheat Sheet

Step	Action
1	**State the goal** — one clear sentence
2	**Give context** — project, language, what's been tried
3	**Set constraints** — format, length, what to avoid
4	**Add examples** — show what good output looks like
5	**Ask for step-by-step reasoning** if the problem is complex
6	**Refine** — treat the first response as a draft

That's the entire skill in 6 steps. Everything in the Workbook is a deeper dive into one of them.

Ready? Continue to the Hands-On Workbook section to start with Module 1.

SECTION 3

Hands-On Workbook

Prompt Engineering — Complete Hands-On Workbook

> ***What you'll do**: Work through 36 hands-on exercises that take you from writing your first good prompt to building reusable templates, constructing a meta-prompt, mastering advanced strategies, getting the most from your AI tools, and automating real client work. **Prerequisites**: Access to at least one AI coding tool — **GitHub Copilot Chat**, **Windsurf (Cascade)**, or **Claude Code**. No prior prompt-engineering experience required. **Time**: ~5–7 hours total (work at your own pace — each module is self-contained, and you can spread it across days/weeks)*

How This Workbook Is Organized

Modules 1–4 are tool-agnostic — the concepts work in any AI coding tool. Modules 5–7 cover tool-specific features for each platform. Do the module(s) that match the tool(s) you have access to.

Module	What You'll Learn	Exercises	Time
1 — Prompt Foundations	Core techniques that make any prompt better	1–5	~40 min
2 — Reusable Templates	Build fill-in-the-blank templates for common tasks	6–10	~40 min
3 — Build Your Own Meta-Prompt	Construct a "prompt that generates prompts"	11–20	~75 min
4 — Advanced Strategies	Iterative refinement, multi-persona, constraint-based	21–23	~30 min
5 — GitHub Copilot	Context references, slash commands, custom instructions	24–26	~30 min
6 — Windsurf (Cascade)	Codebase-aware prompting, rules, workflows	27–28	~25 min
7 — Claude Code	CLAUDE.md, skills, hooks, MCP servers	29–31	~35 min
8 — Automating Client Work	Identify tasks, build client prompts, test end-to-end	32–34	~40 min
9 — Putting It All Together	End-to-end challenge and team prompt library	35–36	~45 min

> ***Don't have a real task to practice with?** Each exercise asks you to use a real problem from your work. If you don't have one, see the **Practice Scenarios** section near the end of this guide*

for 10 realistic examples you can use instead.

How the Concepts Map Across Tools

Every tool has its own names for the same underlying ideas. This table is your Rosetta Stone:

Concept	GitHub Copilot	Windsurf	Claude Code
Project instructions (always-on rules for every conversation)	`.github/copilot-instructions.md`	`.windsurf/rules/*.md`	`CLAUDE.md` (root or any folder)
Reusable prompt templates (fill-in-the-blank prompts)	`.github/prompts/*.prompt.md`	`.windsurf/workflows/*.md`	Custom skills (`.claude/skills/*.md`)
Trigger a saved prompt	`#<prompt-name>` in chat	`/workflow-name` in chat	`/skill-name` in chat
Give file context	`#file:path` or `#selection`	`@file` or paste path	Automatic (reads files on demand)
Give codebase context	Implicit (was `@workspace`)	`@codebase`	Automatic (indexes repo)
Built-in shortcuts	`/explain`, `/fix`, `/tests`, `/doc`	Built into Cascade flow	`/init`, built-in commands
Automation hooks	—	—	Hooks (pre/post command scripts)
External tool integrations	GitHub ecosystem	Built-in web search	MCP servers

***Key insight**: Once you learn the core prompting skills (Modules 1–4), you're learning the same muscle regardless of tool. The tool-specific modules just teach you where to put things.*

How Each Exercise Works

- **Concept** — a short explanation of the technique and why it matters.
- **Bad** → **Good** — a side-by-side comparison so you can see the difference (where applicable).
- **Your Turn** — a hands-on prompt you'll run in your AI tool.
- **Challenge** — an optional stretch task to deepen your understanding.

Module 1 — Prompt Foundations

These exercises teach you the building blocks that make any prompt — meta-prompt or otherwise — more effective.

Exercise 1 — Goal-First Writing

Concept

Most people write prompts like emails: background first, request buried at the end. LLMs perform better when you **lead with the goal**. The first sentence should answer: *"What do I want back?"*

Bad → Good

Bad prompt:

```
I've been working on this project for a while and we use React with TypeScript.
There's this component called UserTable that fetches data from an API and
renders it. Lately it's been kind of slow. The team has complained about it.
Can you take a look?
```

Good prompt:

```
Fix the performance bottleneck in a React/TypeScript component called UserTable
that re-renders excessively when fetching data from an API.
```

The good version states the goal in the first sentence. Context comes after.

Your Turn

Pick a real task from your current work (or invent one). Write it as a prompt following these rules:

1. Open your AI chat and paste:

```
I'm going to give you two versions of the same prompt. Tell me which one is
better and why, then rewrite the weaker one to match the stronger one's quality.

Version A:
So we have this database and it's PostgreSQL and there are these queries that
run every night as part of a batch job and some of them are really slow and
I think they need indexes or something. Can you help?

Version B:
Optimize three slow PostgreSQL queries in our nightly batch job. The queries
scan the `orders` and `shipments` tables (each ~10M rows) without indexes.
Suggest index strategies and rewrite the queries for better performance.
```

2. Read the AI's analysis. It should confirm Version B is stronger because it leads with the goal.
3. Now write your own prompt for a task you care about. **Start with one sentence that states the goal.** Paste it into the chat and ask:

```
Here is a prompt I wrote. Score it 1–10 on clarity, then rewrite it to be
stronger. Explain every change you made.

[paste your prompt here]
```

Challenge

Rewrite 3 prompts from your recent chat history using goal-first structure. Compare the AI's responses to the originals.

Exercise 2 — Providing Context Effectively

Concept

After the goal, the AI needs **context** — but not all context is equal. Effective context answers three questions:

- What **system/project** is this about?
- What **tech stack** is involved?
- What has **already been tried** or decided?

Irrelevant context (project history, team politics, unrelated files) dilutes the signal.

Bad → Good

Bad context:

```
We started this project last year and it's gone through a lot of changes.
The original dev left and now I'm maintaining it. We use a bunch of stuff.
```

Good context:

```
- Project: E-commerce order service (microservice)
- Stack: Java 17, Spring Boot 3.2, PostgreSQL 15, deployed on AWS ECS
- Current state: The /orders endpoint returns 200 but takes 4+ seconds under load
- Already tried: Added connection pooling (HikariCP), no improvement
```

Your Turn

1. Paste this into your AI chat:

```
I'm going to give you a prompt with poor context. Rewrite it with strong,
structured context. Use bullet points. Remove anything irrelevant. Add any
context you think is missing (mark those with "[FILL IN]" so I know to
provide the real values).

Original prompt:
"We have a Node.js app and it crashes sometimes in production. The logs show
some error about memory. It's been happening for a few weeks. We're on AWS.
Can you figure out what's wrong?"
```

2. Review the AI's rewrite. It should produce structured context with tech details, error specifics, and environment info.

3. Now do it yourself — take a problem you're working on and write the context section using this format:

```
- Project: [name and what it does, one line]
- Stack: [languages, frameworks, versions]
- Current state: [what's happening now]
- Already tried: [what hasn't worked]
- Relevant files: [file names or modules]
```

Paste it into the chat and ask the AI to evaluate whether the context is sufficient.

Challenge

Ask the AI: *"What additional context would you need to solve this problem confidently?"* — then fill in the gaps it identifies.

Exercise 3 — Setting Constraints That Shape the Output

Concept

Constraints tell the AI what the output **should and shouldn't** look like. Without them, the AI picks its own format, length, and style — which may not match what you need. Good constraints cover:

- **Format** — code, bullet list, table, paragraph, JSON, etc.
- **Length** — word count, number of items, "fit on one screen"
- **Audience** — senior dev, junior dev, non-technical stakeholder
- **Exclusions** — what to leave out ("don't change module X", "no new dependencies")

Your Turn

1. Paste this into your AI chat:

```
Here is a prompt with no constraints. Add 4–5 constraints that would make the
output more useful for a senior developer reviewing a pull request.

Original prompt:
"Explain what this function does."

Add constraints for: output format, length, audience, what to focus on, and
what to skip.
```

2. Review the AI's version. It should add things like "explain in 3–5 bullet points," "assume the reader is a senior developer," "focus on side effects and edge cases," etc.
3. Now practice on your own. Paste a simple prompt into the chat and ask:

```
I wrote this prompt but it has no constraints. Suggest 5 constraints I should
add to get a more useful response. For each constraint, explain why it helps.

My prompt: [paste your prompt here]
```

Challenge

Write the same prompt with two different constraint sets — one for a junior developer audience and one for a senior developer audience. Run both and compare how the outputs differ.

Exercise 4 — Few-Shot Prompting (Teaching by Example)

Concept

Few-shot prompting means giving the AI one or more examples of the input → output pattern you want *before* asking it to perform the task. This is one of the most powerful techniques available — it lets you show rather than tell.

- **Zero-shot**: No examples, just instructions.
- **One-shot**: One example.
- **Few-shot**: Two or more examples.

Use few-shot when the desired output format is unusual, when the AI keeps getting the format wrong, or when you need a very specific style.

Your Turn

1. Paste this into your AI chat:

```
I want to write commit messages in a specific format. Here are 3 examples of
the style I want:

Example 1:
Input: Added retry logic to the payment service when Stripe returns a 503
Output: fix(payments): add retry with exponential backoff for Stripe 503 errors

Example 2:
Input: Updated the user table to include a `last_login` timestamp column
Output: feat(db): add last_login column to users table

Example 3:
Input: Removed the deprecated /v1/search endpoint
Output: chore(api): remove deprecated /v1/search endpoint

Now write a commit message in the same style for this change:
Input: Fixed a bug where the email notification was sent twice when a user
registered
```

2. Check the output. It should follow the `type(scope): description` pattern from your examples.

3. Now create your own few-shot prompt. Think of a task where you want a **specific output format** — commit messages, PR descriptions, code comments, error messages, test names, etc. Write 2–3 examples and then ask the AI to generate a new one.

```
I want [type of output] in this specific format. Here are my examples:

Example 1:
Input: [describe the situation]
Output: [show the desired output]

Example 2:
Input: [describe the situation]
Output: [show the desired output]

Now generate the output for:
Input: [your new situation]
```

Challenge

Try the same task with zero-shot (no examples) vs. few-shot (3 examples). Compare the quality of the outputs and note the differences.

Exercise 5 — Chain-of-Thought Prompting

Concept

Chain-of-thought (CoT) prompting asks the AI to show its reasoning step by step before giving a final answer. This dramatically improves accuracy on tasks involving:

- Debugging (tracing through code logic)
- Multi-step problems (architecture decisions, migration plans)
- Anything where the AI might "jump to a conclusion" and get it wrong

The key phrase is: **"Think step by step."**

Your Turn

1. Paste this into your AI chat — first **without** chain-of-thought:

```
What is the output of this code?

function mystery(arr) {
  let result = [];
  for (let i = arr.length - 1; i >= 0; i--) {
    if (arr[i] % 2 === 0) {
      result.push(arr[i] * 2);
    }
  }
  return result;
}

console.log(mystery([1, 2, 3, 4, 5, 6]));
```

2. Note the answer. Now paste the same prompt **with** chain-of-thought:

```
What is the output of this code? Think step by step — trace through each
iteration of the loop, show the value of i, whether the condition is true,
and what gets pushed to result.

function mystery(arr) {
  let result = [];
  for (let i = arr.length - 1; i >= 0; i--) {
    if (arr[i] % 2 === 0) {
      result.push(arr[i] * 2);
    }
  }
  return result;
}

console.log(mystery([1, 2, 3, 4, 5, 6]));
```

3. Compare the two responses. The CoT version should show each loop iteration explicitly, making it easier to verify correctness.
4. Now apply this to a real debugging scenario:

```
I have a bug in my code. Before suggesting a fix, think step by step:
1. Read the code and explain what each section does.
2. Trace the execution with the failing input.
3. Identify where the actual behavior diverges from expected behavior.
4. Only then suggest a minimal fix.

[paste your buggy code here]

Failing input: [describe]
Expected output: [describe]
Actual output: [describe]
```

Challenge

Try adding "Think step by step" to 3 different types of prompts (debugging, architecture, data analysis) and note which ones benefit the most.

■ Module 1 Recap — What You Now Know

After Module 1, you can:

- Lead any prompt with the **goal in the first sentence** (Ex. 1)
- Provide **structured context** that's relevant, not just plentiful (Ex. 2)
- Add **constraints** that shape format, length, audience, and exclusions (Ex. 3)
- Use **few-shot examples** to teach the AI a specific output style (Ex. 4)
- Trigger **chain-of-thought reasoning** for debugging and multi-step problems (Ex. 5)

These five techniques are the foundation. Every other module builds on them.

Module 2 — Building Reusable Prompt Templates

> *Now that you know the core techniques, you'll build **reusable templates** that anyone on the team can fill in and get consistent results — no prompt-engineering knowledge required.*

One rule for every template in this module: make it bidirectional

Every template you build in this module must end with a `CLARIFYING QUESTIONS` section. Without it, AI silently guesses whenever input is ambiguous — and guesses produce subtly wrong output. With it, AI asks you 1–3 targeted questions before committing to an answer, and the interaction feels like a conversation instead of a one-way command. Each exercise below asks AI to include this block.

Exercise 6 — Build a Bug Fix Template

Concept

Bug fixes are one of the most common AI tasks. A good bug-fix template ensures the AI always gets: the symptom, the expected behavior, the tech stack, what's been tried, and output constraints.

Your Turn

1. Paste this into your AI chat:

```
Create a reusable prompt template for asking AI to help fix a bug. The template
should have fill-in-the-blank placeholders using [SQUARE BRACKETS]. It must
include these sections:

- GOAL: one-sentence bug description
- CONTEXT: language, framework, file/module, what the code should do
- SYMPTOM: exact error message or unexpected behavior
- EXPECTED BEHAVIOR: what should happen instead
- ALREADY TRIED: previous attempts that didn't work
- CONSTRAINTS: things the fix must not change
- OUTPUT FORMAT: what the AI's response should look like
- CLARIFYING QUESTIONS: tell the AI that if anything above is ambiguous
  or missing context, it must ask 1–3 targeted questions BEFORE
  proposing a fix — not guess silently

After the template, write a short example showing the template filled in for a
real bug (e.g., a NullPointerException in a Spring Boot service).

Output ONLY the template and the filled-in example.
```

2. Review the template. Check that:
   - [ ] Every section has a `[PLACEHOLDER]`
   - [ ] The output format section specifies root-cause analysis + code fix
   - [ ] The **CLARIFYING QUESTIONS** section is present and makes the AI ask rather than assume
   - [ ] The example is realistic and complete
3. **Save the template** in a file called `templates/bug-fix.md` in your repo.

Challenge

Test the template by filling it in with a real bug from your project, then paste the filled-in version into a new AI chat and evaluate the response quality.

Exercise 7 — Build a Code Generation Template

Concept

When asking AI to write code, vague prompts produce vague code. A template forces you to specify inputs, outputs, constraints, and coding standards every time.

Your Turn

1. Paste this into your AI chat:

```
Create a reusable prompt template for asking AI to generate code. It should
include these sections with [PLACEHOLDER] values:

- GOAL: what the code should do
- LANGUAGE/FRAMEWORK: e.g., Python 3.12 / FastAPI
- INPUTS: what data or parameters the code receives
- OUTPUTS: what it returns or produces
- CONSTRAINTS: performance requirements, coding standards, libraries
  allowed/forbidden, backward compatibility needs
- EDGE CASES: specific scenarios the code must handle
- OUTPUT FORMAT: working code with inline comments, plus a brief explanation
  of design decisions
- CLARIFYING QUESTIONS: tell the AI to ask 1–3 questions first when
  design choices are ambiguous (library A vs. B, sync vs. async, strict
  vs. lenient). When a real trade-off exists, the AI should present
  options with brief pros/cons and let the user choose before writing code.

Also include a filled-in example for: "Write a Python function that validates
and parses a CSV upload, returning structured data or detailed error messages."

Output ONLY the template and the filled-in example.
```

2. Review and save as `templates/code-gen.md`.

Exercise 8 — Build a Code Review Template

Concept

AI-assisted code review works best when you tell the AI **what to focus on** — otherwise it gives generic feedback. This template targets specific quality dimensions.

Your Turn

1. Paste this into your AI chat:

```
Create a reusable prompt template for asking AI to review code. It should
include:

- GOAL: what kind of review (quality, performance, security, readability, etc.)
- CODE: placeholder for pasting the code
- CONTEXT: what the code does, where it fits in the system
- FOCUS AREAS: specific concerns (e.g., error handling, SQL injection, N+1
  queries, memory leaks, naming conventions)
- KNOWN CONSTRAINTS: things that can't change (API contract, backward compat)
- OUTPUT FORMAT: issues listed by severity (critical → minor), with suggested
  fixes and code snippets
- CLARIFYING QUESTIONS: tell the AI to ask up to 3 questions when
  context needed to judge severity is missing (is this hot path?
  user-facing? public API?) — before critiquing.

Include a filled-in example reviewing a REST controller for security issues.

Output ONLY the template and the filled-in example.
```

2. Review and save as `templates/code-review.md`.

Exercise 9 — Build a Documentation Template

Concept

Documentation is tedious to write by hand but easy for AI — if you specify the audience, format, and scope clearly. This template ensures consistent docs across the team.

Your Turn

1. Paste this into your AI chat:

```
Create a reusable prompt template for asking AI to write documentation. Include:

- GOAL: what to document (function, API endpoint, architecture, onboarding guide)
- CONTEXT: project or system it belongs to
- AUDIENCE: who will read it (junior dev, senior dev, non-technical stakeholder,
  new team member)
- SCOPE: what to include and what to leave out
- FORMAT: paragraph, bullet list, table, README structure, JSDoc/Javadoc, etc.
- LENGTH: approximate word count or section count
- TONE: technical, conversational, formal
- CLARIFYING QUESTIONS: tell the AI to ask when the audience's
  background or desired depth is unclear. When multiple reasonable
  framings exist (analogy-first vs. technical-first, top-down vs.
  bottom-up), propose 2 options and let the user pick.

Include a filled-in example for: "Document the authentication flow for a new
developer joining the team."

Output ONLY the template and the filled-in example.
```

2. Review and save as `templates/documentation.md`.

Challenge

Use the documentation template to generate docs for something real in your project, then have a teammate read it and give feedback.

Exercise 10 — Have AI Generate a Template for Any Task

Concept

You don't need to hand-craft every template. You can ask the AI to **generate a template for any task type**, then review and refine it. This is a meta-skill — prompting AI to create the prompts you'll use later.

Your Turn

1. Think of a task type your team does frequently that isn't covered by the templates above. Examples:
   - Writing unit tests
   - Creating migration scripts
   - Writing PR descriptions
   - Designing API contracts
   - Writing incident post-mortems
2. Paste this into your AI chat:

```
Create a reusable prompt template for the following task type: [YOUR TASK TYPE].

Requirements:
- Use [PLACEHOLDER] syntax for all variable parts.
- Include sections for: GOAL, CONTEXT, INPUTS, CONSTRAINTS, and OUTPUT FORMAT.
- Add any task-specific sections that make sense (e.g., EDGE CASES for testing,
  SEVERITY for incidents).
- Always end with a CLARIFYING QUESTIONS section that tells the AI to
  ask 1–3 targeted questions before producing output when any input is
  ambiguous, and to surface options-with-pros/cons when a real trade-off
  exists. The goal is bidirectional interaction, not silent assumptions.
- After the template, show it filled in with a realistic example.
- Keep the template concise — someone should be able to fill it in under 2
  minutes.

Output ONLY the template and the filled-in example.
```

3. Review the template. Ask the AI to improve it:

```
Here is the template you generated. Suggest 3 ways to make it more specific
and useful. For each suggestion, give me the exact text to add or change.
```

4. Save the final version in your `templates/` folder.

■ Module 2 Recap — What You Now Know

After Module 2, you can:

- Build a **bug-fix template** that captures symptom, expected behavior, constraints, and asks AI to clarify ambiguity (Ex. 6)
- Build a **code-generation template** that anchors design choices and surfaces trade-offs (Ex. 7)
- Build a **code-review template** that targets specific quality dimensions (Ex. 8)
- Build a **documentation template** that shapes audience, format, and tone (Ex. 9)
- Have AI **generate templates for you** for any new task type (Ex. 10)

You should now have at least 4 saved template files (`templates/bug-fix.md`, `code-gen.md`, `code-review.md`, `documentation.md`) that anyone on the team can fill in to get consistent results.

Module 3 — Build Your Own Meta-Prompt

*A **meta-prompt** is a prompt that turns the AI into a prompt engineer. You paste it into a chat, the AI interviews you about your problem, and then it generates a high-quality prompt you can use in a separate conversation. In this module you'll build one from scratch — by prompting an LLM to write each section.*

***This is the longest module in the guide** (10 exercises, ~60–75 min). There's a midpoint recap after Exercise 15 — feel free to take a break there.*

***Two paths**: this module teaches you the generic meta-prompt architecture by building each part. The **Build Your Own Meta-Prompt** section (Section 4 of this guide) takes a different angle — it walks you through building a meta-prompt for one specific recurring task. Doing both is great; if you're short on time, this module is the foundation, Section 4 is the real-world application.*

***Reference implementation**: a polished, ready-to-use meta-prompt is provided in the **Meta-Prompt File** section (Section 6 of this guide). You can peek at it any time during this module to compare against your draft.*

Before you start

Create a blank file called `my-meta-prompt.md` in your repo. Leave it open — you'll paste into it after each exercise.

***Note on file location**: `my-meta-prompt.md` is a **temporary working location** — a scratchpad where you'll build the file as you progress through the module. In Exercise 20 you'll move it to its **final, tool-specific home** so the team (and your AI tool) can use it natively: - **GitHub Copilot** → `.github/prompts/meta-prompt.prompt.md` - **Windsurf** → `.windsurf/workflows/meta-prompt.md` - **Claude Code** →*

> `.claude/skills/meta-prompt.md` *Don't worry about that yet — just keep it as* `my-meta-prompt.md` *for now and you'll relocate it at the end.*

Exercise 11 — Define the Role and Purpose

Why this matters

Every good prompt starts by telling the AI **who it is** and **what it should do**. This sets the tone and scope for everything that follows. Without a clear role, the AI may try to solve your problem directly instead of helping you craft a prompt.

What to do

Open GitHub Copilot Chat and paste this prompt:

```
I'm building a reusable "meta-prompt" — a set of instructions I can paste into
any AI chat so it becomes a prompt engineer that helps me write better prompts.

Right now I just need the opening section. Write me a short paragraph (3–5
sentences) that I will place at the top of my meta-prompt file. It should:

1. Tell the AI its role: "You are an expert prompt engineer."
2. Tell it its sole purpose: to help the user create a detailed, well-structured
   prompt that will be used in a SEPARATE AI conversation.
3. Make it clear that the AI must NOT solve the user's actual problem — only
   produce the prompt.

Write it in a direct, instructional tone. Use second person ("You are…",
"Your job is…"). Output only the paragraph, no extra commentary.
```

Expected result

You should get a short paragraph that sounds something like:

> *You are an expert prompt engineer. Your sole job is to help the user create a detailed, well-structured prompt that they will then use in a separate AI conversation to solve their actual problem. You must not attempt to solve the user's problem yourself — only produce the prompt.*

Save it

1. Copy the AI's output.
2. Open `my-meta-prompt.md`.
3. Add a heading and paste:

```
# My Meta-Prompt

## SYSTEM INSTRUCTIONS

[paste the paragraph here]
```

Exercise 12 — Create the Clarifying-Questions Protocol

Why this matters

The biggest reason prompts fail is **missing context**. If your meta-prompt just says "generate a prompt," the AI will guess and fill in gaps with assumptions. Instead, we instruct it to **ask questions first** — like a consultant doing a discovery interview — so the generated prompt is grounded in real information.

What to do

Paste this into your AI chat:

```
I'm continuing to build my meta-prompt file. I already have the role/purpose
section. Now I need the "clarifying questions" section.

Write instructions that tell the AI to ask the user clarifying questions BEFORE
generating a prompt. The questions should be asked in 3 rounds:

Round 1 — The Basics (3 questions):
- What problem is the user trying to solve?
- What type of task is it? (bug fix, code generation, code review, docs, etc.)
- What does "done" look like?

Round 2 — Context (3 questions):
- What languages, frameworks, or tools are involved?
- What has already been tried?
- Are there relevant files, error messages, or code snippets?

Round 3 — Constraints & Preferences (3 questions):
- Are there constraints? (don't change X, no new dependencies, etc.)
- What format should the final answer be in?
- Who is the audience?

Requirements for the instructions:
- Tell the AI to ask ONE round at a time and wait for the user's answers before
  continuing to the next round.
- After each round, the AI should briefly summarize what it has learned so the
  user can confirm or correct.
- If any answer is vague, the AI should ask a follow-up question before moving on.

Write this as a numbered set of instructions I can paste directly into my
meta-prompt file under a heading called "PHASE 1 — UNDERSTAND THE PROBLEM".
Output only the instructions, no extra commentary.
```

Expected result

You should get a structured set of instructions with 3 rounds of questions plus rules about summarizing and follow-ups.

Review checklist

- [ ] Does it tell the AI to ask questions **before** generating anything?
- [ ] Are the questions organized into clear rounds?
- [ ] Does it require a summary after each round?
- [ ] Does it handle vague answers?

Save it

Paste the output into `my-meta-prompt.md` under the system instructions you added in Exercise 11.

Exercise 13 — Add the Confirmation Step

Why this matters

Before the AI generates a prompt, it should **play back its understanding** and get explicit confirmation. This prevents misunderstandings from silently propagating into the generated prompt. It's the same principle as reading back an order at a restaurant.

What to do

Paste this into your AI chat:

```
Next section of my meta-prompt. After the AI finishes asking all clarifying
questions, it should present a structured summary and ask the user to confirm
before generating the prompt.

Write instructions for a section called "PHASE 2 — CONFIRM UNDERSTANDING" that
tell the AI to:

1. Present a summary in this exact format:

   HERE IS WHAT I UNDERSTAND:
   - Problem: [one sentence]
   - Task type: [category]
   - Goal: [desired outcome]
   - Tech stack: [languages/frameworks/tools]
   - Constraints: [list]
   - Output format: [format]
   - Audience: [who will read the output]

2. Ask: "Is this correct? Would you like to change or add anything before I
   generate the prompt?"

3. Only proceed to prompt generation once the user explicitly confirms.

Write this as a short, numbered instruction block. Output only the instructions.
```

Expected result

A concise instruction block that describes the confirmation step with the summary template.

Save it

Paste the output into `my-meta-prompt.md` right after the Phase 1 section.

Exercise 14 — Define the Output Prompt Template

Why this matters

This is the core of your meta-prompt: the **structure of the prompt that the AI will generate**. By defining labeled sections (ROLE, GOAL, CONTEXT, etc.), you ensure every generated prompt is consistent, complete, and easy for the next LLM to parse.

What to do

Paste this into your AI chat:

```
Next section of my meta-prompt. After the user confirms the summary, the AI
should generate a prompt using a specific template structure.

Write instructions for a section called "PHASE 3 — GENERATE THE PROMPT" that
tell the AI to produce a prompt with these labeled sections:

- ROLE: A specific role relevant to the user's task
- GOAL: One clear sentence describing what the AI should accomplish
- CONTEXT: Project/system description, tech stack, background, what's been tried
- INPUTS: Any code, error messages, data, or files the AI needs
- CONSTRAINTS: A bulleted list of constraints
- OUTPUT FORMAT: Exactly how the AI should structure its response
- ADDITIONAL INSTRUCTIONS: Step-by-step thinking, assumption handling, etc.

Requirements:
- The generated prompt should be 100–400 words — concise but complete.
- The AI should use plain, direct language in the generated prompt.
- All section labels should be in ALL CAPS so the receiving LLM can parse them.

Write this as a clear instruction block with the template structure shown as an
example. Output only the instructions.
```

Expected result

Instructions that include the template skeleton with all 7 labeled sections and guidance on length/tone.

Review checklist

- [ ] Are all 7 sections present?
- [ ] Is there a word-count guideline?
- [ ] Does it specify plain, direct language?
- [ ] Are sections labeled in ALL CAPS?

Save it

Paste the output into `my-meta-prompt.md` after Phase 2.

Exercise 15 — Add the Review & Refine Loop

Why this matters

The first draft of a generated prompt is rarely perfect. By adding a review step, you give the user a chance to iterate — adjusting wording, adding constraints, or changing the level of detail — before they take the prompt to a new conversation.

What to do

Paste this into your AI chat:

```
Last main section of my meta-prompt. After the AI presents the generated prompt,
it should offer to refine it.

Write instructions for a section called "PHASE 4 — REVIEW & REFINE" that tell
the AI to:

1. Present the generated prompt to the user.
2. Ask: "Would you like me to adjust anything — wording, structure, constraints,
   or level of detail?"
3. Iterate until the user is satisfied.

Keep it short — 3–5 sentences max. Output only the instructions.
```

Save it

Paste the output into `my-meta-prompt.md` after Phase 3.

■ Module 3 Halfway Recap — Take a Breath

You're 5 of 10 exercises into the longest module in the guide. Here's what you've built so far:

Phase	Section in your meta-prompt	What it does
Setup	System Instructions	Tells the AI it's a prompt engineer (Ex. 11)
Phase 1	Clarifying Questions Protocol	AI interviews the user in 3 rounds (Ex. 12)
Phase 2	Confirmation Step	AI summarizes its understanding before generating (Ex. 13)
Phase 3	Output Prompt Template	The structure of the prompt the AI will produce (Ex. 14)
Phase 4	Review & Refine Loop	AI offers to iterate on the generated prompt (Ex. 15)

The **core flow** of your meta-prompt is now complete. The remaining 5 exercises (16–20) add safety rails, an example conversation, validation, customization, and sharing — important polish, but the engine is built.

> ***Tip**: If you want to see what a polished, finished version looks like, peek at the **Meta-Prompt File** section (Section 6 of this guide). Comparing your draft against it can be illuminating — but try to finish your own first so you don't bias yourself.*

Stretch, refill your coffee, and continue when ready. The next 5 exercises are shorter than what you just did.

Exercise 16 — Add Guardrails

Why this matters

Guardrails are rules that keep the AI on track. Without them, the AI might skip questions, make assumptions, or write a bloated 1,000-word prompt. Guardrails act as a safety net for consistent behavior.

What to do

Paste this into your AI chat:

```
I need a "GUARDRAILS" section for my meta-prompt. Write 5–6 bullet-point rules
that the AI must always follow. Include rules like:

- Do not skip the clarifying questions — always ask before generating.
- Do not make assumptions — if something is unclear, ask.
- Keep the generated prompt concise (100–400 words).
- Use plain, direct language.
- Use labeled sections in the generated prompt.
- Do not solve the user's actual problem — only produce the prompt.

Write these as a bulleted list under the heading "GUARDRAILS". Output only the
list.
```

Save it

Paste the output into `my-meta-prompt.md` after Phase 4.

Exercise 17 — Add an Example Conversation (Optional but Recommended)

Why this matters

An example conversation shows the AI **exactly** what the interaction should look like from start to finish. LLMs perform better when they can pattern-match against a concrete example. This is called **few-shot prompting** — a technique you learned in Exercise 4.

What to do

Paste this into your AI chat:

```
Write a short example conversation that demonstrates how my meta-prompt should
work end to end. The example should show:

1. The AI asking Round 1 questions.
2. A user answering (use a realistic scenario — e.g., a React performance issue).
3. The AI asking Round 2 questions.
4. The user answering.
5. The AI asking Round 3 questions.
6. The user answering.
7. The AI presenting the confirmation summary.
8. The user confirming.
9. The AI outputting the final generated prompt.

Keep each exchange to 2–3 sentences. Format it as a blockquoted conversation
using "> **AI**:" and "> **User**:" prefixes. Output only the example.
```

Save it

Paste the output into `my-meta-prompt.md` under a heading called "EXAMPLE CONVERSATION FLOW" at the bottom of the file.

Exercise 18 — Test Your Meta-Prompt

Why this matters

The only way to know if your meta-prompt works is to **use it**. This exercise has you start a fresh conversation, paste your entire meta-prompt, and walk through it with a real problem.

What to do

1. **Open a new chat** in GitHub Copilot Chat.
2. **Copy the entire contents** of `my-meta-prompt.md`.
3. **Paste it** into the chat and press Enter.
4. The AI should begin by asking you Round 1 questions. **Answer them** using a real problem from your current work (or use a made-up scenario).
5. Walk through all 3 rounds of questions.
6. Check that the AI presents a confirmation summary.
7. Confirm and let it generate a prompt.
8. Review the generated prompt.

Evaluation checklist

- [ ] Did the AI ask clarifying questions **before** generating a prompt?
- [ ] Did it ask questions in rounds (not all at once)?
- [ ] Did it summarize after each round?
- [ ] Did it present a confirmation summary and wait for approval?
- [ ] Is the generated prompt between 100–400 words?

- [ ] Does the generated prompt use labeled sections (ROLE, GOAL, etc.)?
- [ ] Did the AI offer to refine the prompt?

If something didn't work

Go back to the relevant exercise and adjust that section of your meta-prompt. Common fixes:

- **AI skipped questions** → Strengthen the Phase 1 instructions with "You MUST ask questions before generating."
- **AI solved the problem instead** → Add to guardrails: "Under no circumstances should you solve the user's problem."
- **Generated prompt was too long** → Tighten the word-count constraint in Phase 3.
- **Missing sections in the generated prompt** → Make the template in Phase 3 more explicit.

Exercise 19 — Customize for Your Team

Why this matters

The meta-prompt you built is generic. To make it truly useful, you should tailor it to your team's tech stack, common tasks, and preferences.

What to do

Paste this into your AI chat:

```
I have a working meta-prompt and I want to customize it for my team. Here is
some information about our work:

- We primarily work with [YOUR LANGUAGES/FRAMEWORKS — e.g., Java, Spring Boot,
  Angular, PostgreSQL].
- Our most common AI tasks are: [e.g., bug fixes, writing unit tests, code
  reviews, writing documentation].
- Our code must follow [YOUR STANDARDS — e.g., company style guide, specific
  linting rules, accessibility standards].
- We deploy to [YOUR PLATFORM — e.g., AWS, Azure, on-prem].

Suggest 3–5 specific changes I should make to my meta-prompt to tailor it for
this team. For each suggestion, give me the exact text I should add or modify.
```

Fill in the bracketed placeholders with your real information before sending.

Save it

Save the customized version as a new file — for example, `my-meta-prompt-team.md` (or `my-meta-prompt-v2.md`) — instead of overwriting your original generic `my-meta-prompt.md`.

Why keep both?

- The **generic version** is a clean starting point you can re-customize for a different team, project, or client later.

- The **team-specific version** is the one you'll actually use day-to-day and ship to your repo.
- If something goes wrong with the customized one, you can always diff against the original to find what changed.

Exercise 20 — Share and Version Your Meta-Prompt

Why this matters

A meta-prompt that lives only on your machine helps only you. Sharing it with your team multiplies its value.

What to do

Option A — Store in your repo:

```
prompts/
  meta-prompt.md       ← your finished file
  README.md            ← short instructions on how to use it
```

Option B — GitHub Copilot custom instructions: Place shared instructions in your repo:

```
.github/copilot-instructions.md
```

Copilot will automatically include these instructions in every chat. You can reference the meta-prompt here or link to it.

Commit it

```
git add my-meta-prompt.md
git commit -m "Add team meta-prompt for generating AI prompts"
git push
```

Meta-Prompt Recap

Phase	What It Does
System Instructions	Sets the AI's role as a prompt engineer
Phase 1 — Understand	AI asks 3 rounds of clarifying questions
Phase 2 — Confirm	AI summarizes understanding and gets approval
Phase 3 — Generate	AI produces a structured prompt (ROLE, GOAL, CONTEXT, etc.)
Phase 4 — Refine	AI offers to iterate on the generated prompt

Phase	What It Does
Guardrails	Rules that keep the AI focused and concise
Example	A sample conversation the AI can pattern-match against

■ Module 3 Recap — What You Now Know

After Module 3, you can:

- Construct a **meta-prompt** that turns any AI chat into a "prompt engineer for you"
- Define **clarifying-questions protocols** so AI interviews users instead of guessing
- Define a **structured output template** (ROLE, GOAL, CONTEXT, INPUTS, CONSTRAINTS, OUTPUT FORMAT) the AI uses every time
- Add **guardrails** that prevent common failure modes (skipping questions, going off-topic, bloated output)
- **Customize and version** the meta-prompt for your team's specific stack and standards

You should now have `my-meta-prompt.md` (or a customized variant) that you've tested end-to-end. **Next**: if you have a specific recurring task in mind, head to Section 4 (Build Your Own Meta-Prompt) to apply this knowledge to that task.

Module 4 — Advanced Prompting Strategies

> *These techniques go beyond the basics and are useful for complex, multi-step, or high-stakes tasks.*

Exercise 21 — Iterative Refinement (Treat It Like a Draft)

Concept

The first AI response is a **draft, not a final answer**. Iterative refinement means sending follow-up prompts that improve the output incrementally. This is one of the most underused techniques — most people accept the first response or start over.

Effective follow-up patterns:

- *"Make this more concise — cut it to half the length."*
- *"Add error handling for the case where X is null."*
- *"Rewrite this for a non-technical audience."*
- *"This is close but [specific issue]. Fix only that part."*

Your Turn

1. Start a new chat and ask:

```
Write a Python function that fetches data from a REST API, retries on failure,
and caches results for 5 minutes.
```

2. Review the first response. Now send **three follow-up refinements**, one at a time:

```
Follow-up 1: Add type hints and a docstring. Use `httpx` instead of `requests`.
```

```
Follow-up 2: The retry logic should use exponential backoff with a max of 3
retries. Show the updated function only — don't repeat the explanation.
```

```
Follow-up 3: Add a unit test for the retry behavior using pytest and
respx for mocking.
```

3. Compare the final version to the first response. The iterative approach should produce significantly better code.

Key takeaway

Three short follow-ups > one giant prompt. Each follow-up is easier to write and gives you a checkpoint to verify quality.

Exercise 22 — Multi-Persona Prompting

Concept

Multi-persona prompting asks the AI to analyze something from multiple perspectives and then synthesize them. This is useful for:

- Architecture decisions (developer vs. ops vs. security)
- Code reviews (performance vs. readability vs. maintainability)
- Trade-off analysis

Your Turn

1. Paste this into your AI chat:

```
I need to decide whether to use a monorepo or a multi-repo setup for a new
microservices project (4 services, 6 developers, deploying to Kubernetes).

Analyze this decision from three perspectives:

1. As a **senior developer**, evaluate developer experience — code sharing,
   dependency management, and daily workflow.
2. As a **DevOps engineer**, evaluate CI/CD complexity, build times, and
   deployment independence.
3. As a **tech lead**, evaluate long-term maintainability, onboarding, and
   team scalability.

After all three perspectives, write a **synthesis** that weighs the trade-offs
and makes a recommendation with clear reasoning.

Format each perspective as a separate section with a heading.
```

2. Review the output. Each persona should surface different concerns.
3. Now try it with your own decision:

```
I need to decide [YOUR DECISION — e.g., which database, which framework,
build vs. buy, etc.].

Analyze from these perspectives:
1. As a [PERSONA 1], evaluate [DIMENSION 1].
2. As a [PERSONA 2], evaluate [DIMENSION 2].
3. As a [PERSONA 3], evaluate [DIMENSION 3].

Synthesize and recommend.
```

Challenge

Use multi-persona for a code review: have the AI review the same code as a security engineer, then as a performance engineer, then as a readability-focused reviewer.

Exercise 23 — Constraint-Based Generation

Concept

Sometimes the best way to get specific output is to **over-constrain** the prompt. Instead of hoping the AI picks the right format, you define every dimension: structure, length, terminology, inclusions, exclusions.

This is especially useful for outputs that need to match a team standard (PR descriptions, ADRs, incident reports, API docs).

Your Turn

1. Paste this into your AI chat:

```
Write an Architecture Decision Record (ADR) for choosing PostgreSQL over
MongoDB for the order management service.

Constraints:
- Use this exact structure: Title, Status, Context, Decision, Consequences
- Title must start with "ADR-001:"
- Status must be one of: Proposed, Accepted, Deprecated, Superseded
- Context section: exactly 3–5 sentences, no more
- Decision section: one clear sentence starting with "We will use…"
- Consequences section: exactly 3 bullet points — one positive, one negative,
  one neutral
- Total length: under 200 words
- Tone: formal, suitable for a team wiki
- Do not mention any database other than PostgreSQL and MongoDB
```

2. Check that **every constraint** was followed. If any were violated, point them out:

```
You missed these constraints: [list them]. Regenerate and follow ALL of them.
```

3. Now create your own over-constrained prompt for a format your team uses:

```
Write a [TYPE OF DOCUMENT] about [TOPIC].

Constraints:
- Structure: [exact sections]
- Length: [word/sentence/bullet count per section]
- Tone: [formal/conversational/technical]
- Must include: [specific elements]
- Must NOT include: [specific exclusions]
- Terminology: [use X, not Y]
```

Challenge

Write a constraint set so tight that two different AI tools produce nearly identical outputs. This is the gold standard for repeatable prompt engineering.

■ Module 4 Recap — What You Now Know

After Module 4, you can:

- Use **iterative refinement** — three short follow-ups to improve output rather than one giant prompt (Ex. 21)
- Use **multi-persona prompting** to surface trade-offs in architecture and design decisions (Ex. 22)
- Use **constraint-based generation** to force outputs that match team standards exactly (Ex. 23)

These are power-user techniques. You'll reach for them when basic prompts don't quite cut it.

Module 5 — GitHub Copilot

> ***Do this module if you have access to GitHub Copilot.*** *Skip it (or come back later) if you don't. These exercises teach you how to get the most out of Copilot Chat's unique features.*

Exercise 24 — Context References: `#file`, `#selection`, and Workspace Awareness

Concept

Copilot Chat lets you inject context automatically using special references:

- `#file:<filename>` — references a specific file
- `#selection` — references the code you have highlighted in the editor
- **Workspace context** — Copilot Chat now reads relevant project context implicitly. You no longer need to prefix prompts with `@workspace`; just ask repo-wide questions and Copilot will look across your project.

> ***Heads-up about*** `@workspace`*: Older Copilot versions required* `@workspace` *to enable repo-wide context. That participant is no longer needed in current Copilot for VS Code — workspace awareness is built in. Some teams have reported that* `#workspace` *(with* `#`*) behaves differently depending on the underlying model (GPT vs. Claude). Tool features evolve quickly; verify the current behavior in your team's chat or Copilot's documentation.*

These references are far more effective than pasting code into the chat manually. They keep the chat clean, provide accurate context, and update automatically if the file changes.

Your Turn

1. Open a file in your editor that has a function you want explained.
2. **Highlight** the function, then type in Copilot Chat:

```
#selection Explain what this function does, what edge cases it handles, and
what edge cases it misses. Format as a bullet list.
```

3. Now try a workspace-wide question (no special prefix needed):

```
What authentication mechanism does this project use? List the relevant
files and explain the flow step by step.
```

4. Try referencing a specific file:

```
#file:package.json Are there any outdated or vulnerable dependencies? List
them with the current version and the recommended version.
```

5. Combine references with a template you built in Module 2:

```
#file:src/services/orderService.ts

GOAL: Review this file for error handling issues.
CONTEXT: This is a microservice that processes customer orders.
FOCUS AREAS: Missing try/catch blocks, unhandled promise rejections,
  generic catch blocks that swallow errors.
OUTPUT FORMAT: List issues by severity (critical → minor) with line
  numbers and suggested fixes.
```

Key takeaway

Always use context references instead of pasting code. Combine them with your templates for powerful, context-aware prompts.

Exercise 25 — Slash Commands: Built-In Prompt Shortcuts

Concept

Copilot Chat has built-in slash commands that trigger specialized behaviors. These are shortcuts — each one is essentially a well-crafted prompt that GitHub has built in for you:

Command	What It Does
`/explain`	Explains the selected code
`/fix`	Suggests a fix for problems in the selected code
`/tests`	Generates unit tests for the selected code
`/doc`	Generates documentation comments

You can combine these with your own instructions to get better results.

Your Turn

1. **Select a function** in your editor, then try each slash command:

```
/explain — focus on what this function assumes about its inputs and what
happens when those assumptions are violated.
```

```
/fix — there's a potential null reference on line [X]. Fix only that issue
and explain the root cause.
```

```
/tests — use [your test framework, e.g., Jest/pytest/JUnit]. Cover:
1. Happy path with valid input
2. Edge case with empty input
3. Error case with invalid input
Write each test with a descriptive name that explains the scenario.
```

```
/doc — write JSDoc/Javadoc for this function. Include @param, @returns,
@throws, and a one-line description. Skip obvious parameters.
```

2. Notice that adding instructions **after** the slash command improves the output significantly compared to using the slash command alone.

Key takeaway

Slash commands are a starting point, not the final prompt. Always add context and constraints after the command to shape the output.

Exercise 26 — Custom Instructions: `.github/copilot-instructions.md`

Concept

You can create a `.github/copilot-instructions.md` file in your repo that Copilot automatically includes in **every** chat conversation. This is where you encode your team's standards so nobody has to remember to include them manually.

What a minimal `.github/copilot-instructions.md` looks like

```
# Copilot Instructions

## Stack
- TypeScript (strict), React 18, Node.js 20, PostgreSQL 15
- Tests: Jest + React Testing Library
- Lint/format: ESLint + Prettier (run on save)

## Conventions
- Components: PascalCase, one component per file
- Hooks: `useXxx` naming, in `src/hooks/`
- API calls: go through `src/api/client.ts`, never raw fetch
- Use `import type` for type-only imports

## Don't suggest
- `any` (use `unknown` and narrow)
- `console.log` in production code
- Deprecated lifecycle methods or class components
- Inline styles (use CSS modules)

## Always include
- Error boundaries around new top-level components
- Loading and error states for async UI
```

Your Turn

1. Paste this into Copilot Chat:

```
Write a `.github/copilot-instructions.md` file for a team that works with
[YOUR TECH STACK — e.g., TypeScript, React, Node.js, PostgreSQL]. It should
tell Copilot to:

- Always follow [YOUR CODING STANDARDS — e.g., Airbnb style guide]
- Use [YOUR PREFERRED PATTERNS — e.g., repository pattern, dependency injection]
- Write tests using [YOUR TEST FRAMEWORK — e.g., Jest, pytest, JUnit]
- Format code according to [YOUR FORMATTER/LINTER — e.g., Prettier, ESLint]
- Never suggest [THINGS TO AVOID — e.g., any, console.log in production code]
- When generating code, always include error handling for [COMMON CASES]
- Use [YOUR PROJECT'S NAMING CONVENTIONS — e.g., camelCase for variables,
  PascalCase for components]

Keep it under 30 lines. Make each instruction direct and specific.
Output the file content only.
```

2. Fill in the placeholders with your real team info before sending.
3. Review the output, then create the file:

```
mkdir -p .github
# paste the content into .github/copilot-instructions.md
```

4. **Test it**: Open a new Copilot Chat and ask it to write some code. Check that the generated code follows the instructions you just set.

Key takeaway

Custom instructions are the highest-leverage Copilot feature for teams. Set them once, and every team member gets consistent, standards-compliant suggestions automatically.

Challenge

Compare the output of the same prompt **before** and **after** adding custom instructions. Note the differences in code style, patterns, and conventions.

■ Module 5 Recap — What You Now Know

After Module 5, you can:

- Use `#file` and `#selection` to give Copilot real code context instead of pasting (Ex. 24)
- Combine **slash commands** (`/explain`, `/fix`, `/tests`, `/doc`) with extra instructions for sharper output (Ex. 25)
- Set up `.github/copilot-instructions.md` so every chat in a project follows your team's standards automatically (Ex. 26)

You should now have a `.github/copilot-instructions.md` committed in at least one repo.

Module 6 — Windsurf (Cascade)

> ***Do this module if you have access to Windsurf.*** *Skip it (or come back later) if you don't. Windsurf's Cascade can read files, search your codebase, and make edits directly — your prompts become action-oriented, not just question-oriented.*

Exercise 27 — Codebase-Aware Prompting with Cascade

Concept

Cascade has deep codebase awareness built in. Unlike tools where you manually reference files, Cascade can **read, search, and edit** your codebase autonomously. This means your prompts can be higher-level — describe the outcome, and Cascade figures out which files to touch.

Key context references:

- `@file` — reference a specific file by path
- `@codebase` — give Cascade access to search the entire project
- `@web` — let Cascade search the internet for documentation or solutions

> ***Heads-up about*** `@codebase`*: Some Windsurf installations (including certain enterprise / Cap1 setups) don't expose the* `@codebase` *tag explicitly.* ***The good news****: Cascade has codebase awareness built in regardless. If* `@codebase` *isn't recognized in your install, just remove the tag and ask the question directly — the prompts below all work without it. Try the tagged version*

first; if it errors or behaves oddly, fall back to the untagged version.

Your Turn

1. In Windsurf Chat, ask Cascade to explore your project:

```
@codebase Give me a brief summary of what this project does, the main
technologies used, and the entry point file. List the top-level folder
structure.
```

2. Ask it to find patterns across the codebase:

```
@codebase Find all the places in this project where we handle errors or
exceptions. List each file and line number. Are we consistent in our
error-handling approach? If not, describe the inconsistencies.
```

3. Ask it to take action (Cascade will propose changes for your approval):

```
@file:src/services/orderService.ts This function doesn't handle the case
where the API returns a 404. Add error handling that throws a descriptive
OrderNotFoundError. Don't change anything else in the file.
```

4. Combine codebase context with a template you built in Module 2:

```
@file:src/services/paymentService.ts

GOAL: Review this file for security issues.
CONTEXT: This service handles credit card tokenization via Stripe.
FOCUS AREAS: Input validation, secret handling, error messages that might
  leak internal details, missing authentication checks.
OUTPUT FORMAT: Issues by severity (critical → minor) with line numbers
  and suggested fixes.
```

Key takeaway

Cascade's strength is that prompts can be **action-oriented** — you describe what to change, and it reads, edits, and proposes the diff. Use `@codebase` for discovery and `@file` for targeted work.

Exercise 28 — Windsurf Rules and Workflows

Concept

Windsurf has two mechanisms for reusable instructions — they map directly to concepts you already know:

Windsurf Feature	What It Does	Equivalent To
Rules (`.windsurf/rules/*.md`)	Always-on instructions included in every conversation	Copilot's `.github/copilot-instructions.md`
Workflows (`.windsurf/workflows/*.md`)	Saved multi-step prompts triggered by `/name`	Copilot's prompt files, Claude Code skills

Your Turn — Create a Rule

1. Paste this into Windsurf Chat:

```
Write a Windsurf rules file (`.windsurf/rules/team-standards.md`) for a
team that works with [YOUR TECH STACK]. The rule should tell Cascade to:

- Always follow [YOUR CODING STANDARDS]
- Use [YOUR PREFERRED PATTERNS]
- Write tests using [YOUR TEST FRAMEWORK]
- Never suggest [THINGS TO AVOID]

Use the Windsurf rules format: start with a short description line, then
the instructions as markdown. Keep it under 30 lines. Output only the
file content.
```

2. Save the output to `.windsurf/rules/team-standards.md`.

What a minimal Windsurf rule looks like

```
---
description: Team coding standards (always-on)
---

- Use TypeScript strict mode; no `any`.
- React components: function components with hooks only.
- API calls go through `src/api/client.ts`.
- Tests: Jest + React Testing Library; aim for behavioral assertions, not
  implementation details.
- Do not suggest `console.log` in production code.
- Follow Prettier formatting (do not manually format; run the formatter).
```

Your Turn — Create a Workflow

1. Paste this into Windsurf Chat:

```
Write a Windsurf workflow file (`.windsurf/workflows/review-code.md`) that,
when I type `/review-code` in chat, does the following:

1. Asks which file to review.
2. Reads the file using @file.
3. Reviews it for: error handling, performance, security, and readability.
4. Lists issues by severity with suggested fixes.
5. Offers to apply the fixes directly.

Use the Windsurf workflow format with YAML frontmatter (name, description)
followed by markdown steps. Output only the file content.
```

2. Save it and test by typing `/review-code` in a new Windsurf Chat.

What a minimal Windsurf workflow looks like

```
---
name: review-code
description: Review a file for quality, security, and readability
---

# Review Code Workflow

Steps Cascade should follow when this workflow is invoked:

1. Ask the user which file to review. If a file is already open, default to it.
2. Read the file with @file.
3. Review for:
   - Bugs and logic errors
   - Missing error handling
   - Security issues (input validation, secret handling)
   - Readability (naming, function length, comments)
4. Output findings in this format:
   - **Critical**: must-fix issues
   - **Major**: should-fix issues
   - **Minor**: nice-to-have improvements
   For each, include the line number, the issue, and a suggested fix.
5. After the report, ask: "Would you like me to apply any of these fixes?"

If the user's intent is unclear (which file? what to focus on?), ask
1–3 targeted questions before reviewing.
```

3. Now create a workflow for a task your team does frequently:

```
Write a Windsurf workflow for: [YOUR TASK — e.g., "generate unit tests for
a service class"]. It should:

1. [Step 1 — e.g., "Ask which file to test"]
2. [Step 2 — e.g., "Read the file and identify public methods"]
3. [Step 3 — e.g., "Generate tests using JUnit 5 and Mockito"]
4. [Step 4 — e.g., "Write the test file next to the source file"]

Use Windsurf workflow format with YAML frontmatter. Output only the file.
```

Key takeaway

Rules = "always do this" (set once, affects every conversation). Workflows = "do this when I ask" (triggered on demand with `/name`). Together they let you encode team standards **and** automate specific tasks.

Challenge

Create 3 workflows for your team's most common tasks and commit them to `.windsurf/workflows/`. Have a teammate test them.

■ Module 6 Recap — What You Now Know

After Module 6, you can:

- Use `@codebase` and `@file` for Cascade's codebase-aware prompting (with fallback if `@codebase` isn't available in your install) (Ex. 27)
- Create `.windsurf/rules/*.md` for always-on team standards (Ex. 28)
- Create `.windsurf/workflows/*.md` for reusable multi-step prompts triggered with `/name` (Ex. 28)

You should now have at least one rule file and one workflow committed to a `.windsurf/` directory.

Module 7 — Claude Code

> ***Do this module if you have access to Claude Code.*** *Skip it (or come back later) if you don't. Claude Code operates directly in your terminal with full codebase awareness, and its customization system (CLAUDE.md, skills, hooks) is the most programmable of the three tools.*

Exercise 29 — CLAUDE.md: Project Instructions

Concept

`CLAUDE.md` is Claude Code's equivalent of project instructions. Place it at the root of your repo (or in any subfolder) and Claude Code reads it automatically at the start of every conversation.

What makes it different:

- **Hierarchical** — you can have a root `CLAUDE.md` plus folder-level ones (e.g., `src/api/CLAUDE.md`) that add context for that area of the codebase
- **Automatically loaded** — no need to reference it; Claude reads it on startup
- **Markdown format** — same format as everything else you've been writing

What a minimal `CLAUDE.md` looks like

If you've never seen one before, here's a small but realistic example you can use as a reference. Place this at your repo root as `CLAUDE.md`:

```
# Project: order-service

A Spring Boot 3.2 microservice that processes customer orders.
Java 17, PostgreSQL 15, deployed to AWS ECS.

## Coding standards
- Google Java Style; format with `mvn spotless:apply`
- Use constructor injection, never field injection
- Prefer Optional<T> over null returns
- All public methods need Javadoc

## Testing
- JUnit 5 + Mockito; integration tests use Testcontainers
- 80% line coverage minimum; run with `mvn test jacoco:report`

## Common commands
- Build: `mvn clean install`
- Run locally: `mvn spring-boot:run`
- Lint/format: `mvn spotless:check` (use `:apply` to fix)
- Test: `mvn test`

## Things to avoid
- Raw SQL — use JPA repositories
- Catching generic Exception — be specific
- Logging at INFO inside loops

## Project layout
- `src/main/java/com/acme/orders/` — domain code
- `src/main/java/com/acme/orders/api/` — REST controllers
- `src/main/java/com/acme/orders/persistence/` — JPA entities & repos
```

Keep it under ~50 lines. If your project is large, split into folder-level `CLAUDE.md` files instead of inflating the root file.

Your Turn

1. Open Claude Code in your terminal and paste this prompt:

```
Help me write a CLAUDE.md file for the root of my project. Here's the info:

- Project: [NAME — what it does in one sentence]
- Stack: [languages, frameworks, versions]
- Coding standards: [style guide, linter, formatter]
- Test framework: [e.g., Jest, pytest, JUnit] with [coverage requirements]
- Architecture: [e.g., monolith, microservices, monorepo — key patterns]
- Common commands: [build, test, lint, deploy commands]
- Things to avoid: [anti-patterns, deprecated APIs, etc.]

Write it as a concise CLAUDE.md (under 50 lines). Use markdown headers to
organize sections. Include a "Common Commands" section with the exact shell
commands for building, testing, and linting. Output only the file content.
```

2. Save the output as `CLAUDE.md` in your repo root.
3. **Test it**: Start a new Claude Code session in the repo and ask it to write some code. Check that it follows the standards you defined.
4. (Optional) Create a subfolder-level CLAUDE.md:

```
Write a CLAUDE.md for the `src/api/` folder of my project. This folder
contains [DESCRIPTION — e.g., "REST API controllers using Express.js"].
Add instructions specific to this folder:
- [e.g., "All endpoints must validate request body with Zod schemas"]
- [e.g., "Use the ApiError class for error responses, never throw raw errors"]
- [e.g., "Every new endpoint needs an integration test in __tests__/"]

Keep it under 20 lines. Output only the file content.
```

Key takeaway

Think of CLAUDE.md as a briefing document for a new team member — what they need to know before touching code in this project. The hierarchical system lets you add specialized context where it matters.

Exercise 30 — Skills: Reusable Slash Commands

Concept

Skills in Claude Code are reusable prompt templates saved as markdown files. Once created, any team member can trigger them with a slash command (e.g., `/generate-tests`). They're the Claude Code equivalent of Windsurf workflows and Copilot prompt files.

Skills are stored in:

- **Project skills**: `.claude/skills/*.md` (shared with the team via git)
- **Personal skills**: `~/.claude/skills/*.md` (only on your machine)

Each skill file is a markdown document with instructions that Claude Code follows when you invoke it.

What a minimal skill file looks like

Save this as `.claude/skills/generate-tests.md`:

```
---
name: generate-tests
description: Generate unit tests for the currently open file
---

# Generate Tests

When invoked, do the following:

1. Identify the file the user wants tested. If unclear, ask.
2. Read the file and list its public methods/functions.
3. For each one, generate tests covering:
   - The happy path with valid inputs
   - At least one edge case (empty, null, boundary)
   - At least one error case (invalid input, missing dependency)
4. Use the project's test framework (check the project's CLAUDE.md;
   default to JUnit 5 + Mockito for Java, Jest for TS/JS, pytest for Python).
5. Match the style of any existing tests in the same module.
6. Write the test file next to the source file with the conventional
   naming (e.g., `OrderService.java` -> `OrderServiceTest.java`).

Before writing, ask: "Which file should I generate tests for?" if the
user didn't specify one.
```

Then trigger it in Claude Code with `/generate-tests`.

Your Turn

1. Ask Claude Code to help you create a skill:

```
Create a Claude Code skill file (`.claude/skills/review-pr.md`) that,
when invoked, does the following:

1. Asks which branch or PR to review (or defaults to the current branch diff)
2. Reads the changed files using git diff
3. Reviews each file for:
   - Bugs or logic errors
   - Missing error handling
   - Security issues
   - Code style violations based on the project's CLAUDE.md
4. Produces a summary organized by severity (critical → minor)
5. Offers to generate fix suggestions for critical issues

Write it as a markdown file with clear instructions. Include a frontmatter
section with the skill name and description. Output only the file content.
```

2. Save and test: run `/review-pr` in Claude Code.
3. Now create a skill for a task your team does frequently. Use this pattern:

```
Create a Claude Code skill (`.claude/skills/[SKILL-NAME].md`) that:

1. [STEP 1 — what Claude should do first]
2. [STEP 2 — what comes next]
3. [STEP 3 — the main action]
4. [STEP 4 — output or follow-up]

Context:
- This skill will be used for [TASK DESCRIPTION]
- Our stack is [TECH STACK]
- The output should follow [STANDARDS/FORMAT]

Write it as a markdown skill file. Output only the file content.
```

Key takeaway

Skills turn your best prompts into one-command workflows the whole team can use. If you find yourself typing the same multi-step prompt more than twice, make it a skill.

Exercise 31 — Hooks and MCP Servers

Concept

Claude Code has two power features with no equivalent in Copilot or Windsurf:

Hooks — scripts that run automatically before or after Claude Code actions:

- `PreToolUse` — runs before Claude uses a tool (e.g., validate before writing a file)
- `PostToolUse` — runs after a tool completes (e.g., auto-format after code generation)
- `PreCommit` — runs before committing (e.g., run linter, check for secrets)

MCP Servers — external tool integrations that give Claude Code new capabilities (database access, API calls, Jira integration, etc.)

What a minimal `.claude/settings.json` looks like

If you've never seen this file, here's a small but realistic example showing both a hook and an MCP server. Save as `.claude/settings.json` at your repo root:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "npx eslint --fix \"$CLAUDE_FILE_PATH\"",
            "filePattern": "*.{ts,tsx,js,jsx}"
          }
        ]
      }
    ]
  },
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
               "postgresql://user:pass@localhost:5432/mydb"]
    }
  }
}
```

What's happening here:

- The **hook** runs `eslint --fix` automatically every time Claude writes or edits a JavaScript/TypeScript file. No manual reminder needed.
- The **MCP server** gives Claude Code read access to a local Postgres database so it can answer questions like "what columns does the orders table have?" without you pasting schema dumps.

> ***Important***: *The `.claude/settings.json` schema and field names evolve as Claude Code adds features. Treat this example as a starting reference and verify against the current Claude Code documentation before relying on it in production. The Claude Code CLI can also generate or update this file for you (`/init`, or just ask).*

Your Turn — Create a Hook

1. Ask Claude Code:

```
Help me create a Claude Code hook that automatically runs our linter after
any file is written or edited. The hook should:

- Trigger: after any file write (PostToolUse for Write and Edit tools)
- Action: run [YOUR LINT COMMAND — e.g., "npx eslint --fix"] on the
  changed file
- Only run on files matching [PATTERN — e.g., "*.ts", "*.tsx"]
- If the linter finds errors, show them to Claude so it can fix them

Show me the settings.json configuration for this hook and explain each field.
```

2. Review the configuration and add it to your `.claude/settings.json`.
3. **Test it**: Ask Claude Code to write a function with intentionally sloppy formatting. The hook should auto-format it.

Your Turn — Understand MCP Servers

1. Ask Claude Code:

```
Explain MCP (Model Context Protocol) servers in Claude Code. Specifically:
1. What are they and what problem do they solve?
2. Give me 3 practical examples of MCP servers a development team would use
   (e.g., database, Jira, Slack).
3. For one of those examples, show me what the configuration in
   settings.json would look like.
4. How does a team share MCP server configurations?

Keep the explanation under 300 words. I'm a developer, not an AI researcher.
```

2. If your team uses a tool that has an MCP server (Jira, Linear, Postgres, etc.), try setting it up.

Key takeaway

Hooks let you enforce team standards automatically — no discipline required. MCP servers extend what Claude Code can do beyond just reading code. Together, they make Claude Code the most customizable of the three tools.

Challenge

Create a `PreCommit` hook that checks for hardcoded secrets (API keys, passwords) before allowing a commit.

■ Module 7 Recap — What You Now Know

After Module 7, you can:

- Write a `CLAUDE.md` that gives Claude Code project context automatically — at root or per-folder (Ex. 29)
- Create reusable `.claude/skills/*.md` that teammates trigger with `/skill-name` (Ex. 30)
- Configure **hooks** in `.claude/settings.json` to enforce standards automatically (auto-lint, secret checks, etc.) (Ex. 31)
- Understand **MCP servers** and when to set them up for external tool integrations (Ex. 31)

Claude Code is the most customizable of the three tools — you've now seen its full surface area.

Module 8 — Automating Client Work

> *This is where everything comes together with a practical focus: **how to identify tasks in client projects that your AI tool can help automate, and how to build the prompts to do it.** These exercises work with any tool.*

Exercise 32 — Identify Automatable Tasks

Concept

Not every task is a good fit for Copilot. The best candidates are:

- **Repetitive** — you or the team do it frequently across projects
- **Well-defined** — clear inputs, expected outputs, known constraints
- **Code-centric** — involves writing, reviewing, transforming, or documenting code
- **Low-risk to iterate** — a wrong first attempt is cheap to fix

Poor candidates: tasks requiring deep business domain judgment, tasks with ambiguous requirements, or tasks where the output is highly creative and subjective.

Your Turn

1. Think about your current or recent client work. Paste this into Copilot Chat:

```
I work on client software projects. Here are some tasks my team does regularly:

- [TASK 1 — e.g., writing CRUD endpoints for new entities]
- [TASK 2 — e.g., writing unit tests for service classes]
- [TASK 3 — e.g., converting SQL queries to ORM calls]
- [TASK 4 — e.g., writing API documentation from code]
- [TASK 5 — e.g., creating migration scripts for schema changes]
- [TASK 6 — e.g., reviewing PRs for security issues]

For each task, rate it as HIGH / MEDIUM / LOW suitability for automation
with an AI coding assistant (GitHub Copilot Chat — not agent mode, just chat).

For each rating, explain:
- Why it's suitable or not
- What parts could be automated vs. what needs human judgment
- What the prompt would need to include for good results

Format as a table.
```

2. Review the ratings. You should see that tasks with clear structure and repeatable patterns score highest.
3. Pick the **top 2 HIGH-rated tasks**. You'll build prompts for them in the next exercises.

Challenge

Add tasks specific to your client's industry (healthcare compliance checks, financial data validation, etc.) and re-evaluate.

Exercise 33 — Build a Client-Specific Prompt Template

Concept

A client-specific prompt template is just like the templates from Module 2, but tailored to a specific client's tech stack, patterns, and requirements. The goal: any developer on your team can fill it in and get output that matches the client's standards.

Your Turn

1. Pick one of the HIGH-rated tasks from Exercise 32. Paste this into Copilot Chat:

```
I need to create a reusable prompt template that my team can use in GitHub
Copilot Chat to automate the following task for a client project:

Task: [DESCRIBE THE TASK — e.g., "Generate unit tests for Spring Boot
service classes"]

Client context:
- Tech stack: [e.g., Java 17, Spring Boot 3.2, JUnit 5, Mockito, PostgreSQL]
- Coding standards: [e.g., Google Java Style, 80% code coverage minimum]
- Patterns used: [e.g., repository pattern, service-controller layers,
  DTOs for API responses]
- Special requirements: [e.g., all tests must use @Transactional with
  rollback, test database is H2 in-memory]

Create a prompt template with [PLACEHOLDER] syntax that:
1. Is specific enough to produce output matching this client's standards
2. Is generic enough that the team can use it for any class in this project
3. Includes a filled-in example showing the template in use

The template should work when pasted directly into Copilot Chat alongside
a #file reference to the target class.

Output ONLY the template and the filled-in example.
```

2. Review the template. Test it by filling it in for a real class in your project:
   - Use `#file:<path>` in Copilot Chat to reference the target file
   - Paste the filled-in template
   - Evaluate the output against the client's standards
3. If the output isn't right, iterate:

```
The generated output has these issues: [list them].
Update the template to prevent these issues. Add constraints or examples
as needed.
```

4. Save the final template in `templates/client-[project-name]/[task].md`.

Key takeaway

Client-specific templates are your team's biggest time saver. Once a template produces good output, anyone can use it — no prompt engineering knowledge needed.

Exercise 34 — Test and Validate Your Client Prompt

Concept

A prompt template isn't ready until you've tested it against **multiple real inputs** and verified the output is production-quality. This exercise walks you through a validation workflow.

Your Turn

1. Take the template you built in Exercise 33.
2. Fill it in for **3 different inputs** from the same client project (e.g., 3 different service classes, 3 different API endpoints, 3 different bug reports).
3. For each, run it in Copilot Chat using `#file` references and evaluate:

Criterion	Input 1	Input 2	Input 3
Follows client coding standards?	Y/N	Y/N	Y/N
Correct and complete output?	Y/N	Y/N	Y/N
Required manual edits?	none / minor / major	...	...
Time saved vs. manual?	estimate	...	...

4. If any input produced poor output, diagnose why:

```
I used this prompt template: [paste template]
With this input: [paste the input that failed]
The output had these problems: [list them]

What should I change in the template to prevent these problems?
Suggest specific additions or modifications.
```

5. Update the template and re-test until all 3 inputs produce good output.
6. **Document it**: At the top of the template file, add a short note:

```
<!-- Tested against: OrderService, PaymentService, ShippingService -->
<!-- Last validated: [date] -->
<!-- Typical time saved: ~15 min per class -->
```

Key takeaway

A template that works for 1 input is a draft. A template that works for 3+ diverse inputs is ready to share with the team.

■ Module 8 Recap — What You Now Know

After Module 8, you can:

- **Identify which client tasks** are good automation candidates (repetitive, well-defined, code-centric, low-risk to iterate) vs. which aren't (Ex. 32)
- **Build a client-specific prompt template** tailored to a real project's stack and standards (Ex. 33)
- **Validate a template** against multiple real inputs before shipping it to the team (Ex. 34)

You should now have at least one client-specific template tested against 3+ real inputs.

Module 9 — Putting It All Together

Exercise 35 — End-to-End Challenge

Concept

This exercise combines everything you've learned. You'll take a real problem, use the right technique to solve it, and evaluate the quality of the result.

Your Turn

1. **Pick a real task** from your current work. Choose one:
   - A bug you need to fix
   - A feature you need to implement
   - Code you need to review
   - Documentation you need to write
2. **Decide which technique fits**:

Task	Best Technique
Bug fix	Bug-fix template (Ex. 6) + chain-of-thought (Ex. 5)
New feature	Code-gen template (Ex. 7) + iterative refinement (Ex. 21)
Code review	Code-review template (Ex. 8) + multi-persona (Ex. 22)
Documentation	Documentation template (Ex. 9) + audience constraints (Ex. 3)
Architecture decision	Multi-persona (Ex. 22) + constraint-based (Ex. 23)
Client task automation	Client template (Ex. 33) + validation (Ex. 34)
Not sure how to start	Meta-prompt (Ex. 11–20) to generate the prompt for you

3. **Build your prompt** using the matching template from Module 2 (or Module 8 for client work), enhanced with techniques from Modules 1 and 4.
4. **Run it** in your AI tool, using the tool-specific features from Modules 5–7.
5. **Refine** with 2–3 follow-up messages.

6. **Evaluate** the final result:
   - [ ] Did the output match the requested format?
   - [ ] Was the quality high enough to use (or nearly so)?
   - [ ] Did the prompt + follow-ups take less time than doing it manually?

Exercise 36 — Build a Team Prompt Library

Concept

The most productive teams don't start from scratch every time — they maintain a **shared prompt library** that anyone can use. This includes both general templates and client-specific ones.

Your Turn

1. Paste this into your AI tool's chat:

```
I want to create a team prompt library stored in our repo under a `prompts/`
folder. Based on the following common tasks our team performs, suggest a
folder structure and list the template files we should create:

Common tasks:
- [TASK 1 — e.g., fixing bugs]
- [TASK 2 — e.g., writing unit tests]
- [TASK 3 — e.g., code reviews]
- [TASK 4 — e.g., writing API documentation]
- [TASK 5 — e.g., incident post-mortems]

For each template file, write a one-line description of what it does.
Also include a README.md that explains how to use the library.

Output the folder structure and the README content.
```

2. Review the structure and create the files in your repo.
3. Fill in each template using the techniques from this workbook (or use Exercise 10 to have AI generate them).
4. Commit and share with your team:

```
git add prompts/
git commit -m "Add team prompt library with reusable templates"
git push
```

Quick Reference — When to Use What

Technique	Use When…
Goal-first writing	Every prompt — always start with the goal

Technique	Use When…
Structured context	The AI needs project/tech/history details
Constraints	You need a specific format, length, or style
Few-shot examples	The output format is unusual or very specific
Chain-of-thought	Debugging, logic, or multi-step reasoning
Iterative refinement	The first response is close but not right
Multi-persona	Architecture decisions, trade-off analysis
Constraint-based	Team standards, repeatable outputs
Templates	Any task the team does repeatedly
Client templates	Automating repetitive client work
Meta-prompt	When you're not sure how to write the prompt at all
Context references	Always — give the AI real code context, don't paste
Project instructions	Set once per repo, enforces team standards automatically

Where to Put Things (By Tool)

What	Copilot	Windsurf	Claude Code
Team standards	`.github/copilot-instructions.md`	`.windsurf/rules/*.md`	`CLAUDE.md`
Reusable prompts	`.github/prompts/*.prompt.md`	`.windsurf/workflows/*.md`	`.claude/skills/*.md`
Templates	`prompts/templates/*.md`	`prompts/templates/*.md`	`prompts/templates/*.md`

What's Next

- **See the Quick Reference section** for a quick lookup of all patterns and techniques.
- **See the Meta-Prompt File section** for a polished, ready-to-use meta-prompt you can compare yours against.
- **Set up project instructions** for every active project (whichever tool you use).
- **Build client-specific templates** for your top 3 repetitive tasks and share them with the team.
- **Practice daily** — the more you prompt, the faster you get.

Good prompts are a skill, not a talent. Every exercise you complete makes the next prompt better.

SECTION 4

Build Your Own Meta-Prompt

Build Your Own Meta-Prompt with AI

> ***What this is**: A focused, hands-on path that shows you how to use AI (GitHub Copilot Chat) to help you build a **reusable prompt-builder** — a single prompt file you can hand back to AI anytime in the future, and it will produce tailored, high-quality prompt templates for your specific use cases and deliverables. **Time**: ~60–90 minutes **Prerequisite**: Access to GitHub Copilot Chat (or any AI chat tool).*

How This Section Relates to Module 3

This section and **Module 3 of the Workbook** both teach you to build meta-prompts — but with different angles:

	Module 3 (Workbook)	This Section
Approach	Learn-by-building — you construct each piece (system instructions, clarifying questions, output template)	Use-case-driven — you pick one recurring task and tailor a meta-prompt to it
Output	A generic meta-prompt that works for any task type	A specialized meta-prompt for one specific recurring task
Best for	Understanding the architecture of a meta-prompt	Applying the architecture to a real, specific need
Format	10 short exercises	6 longer steps

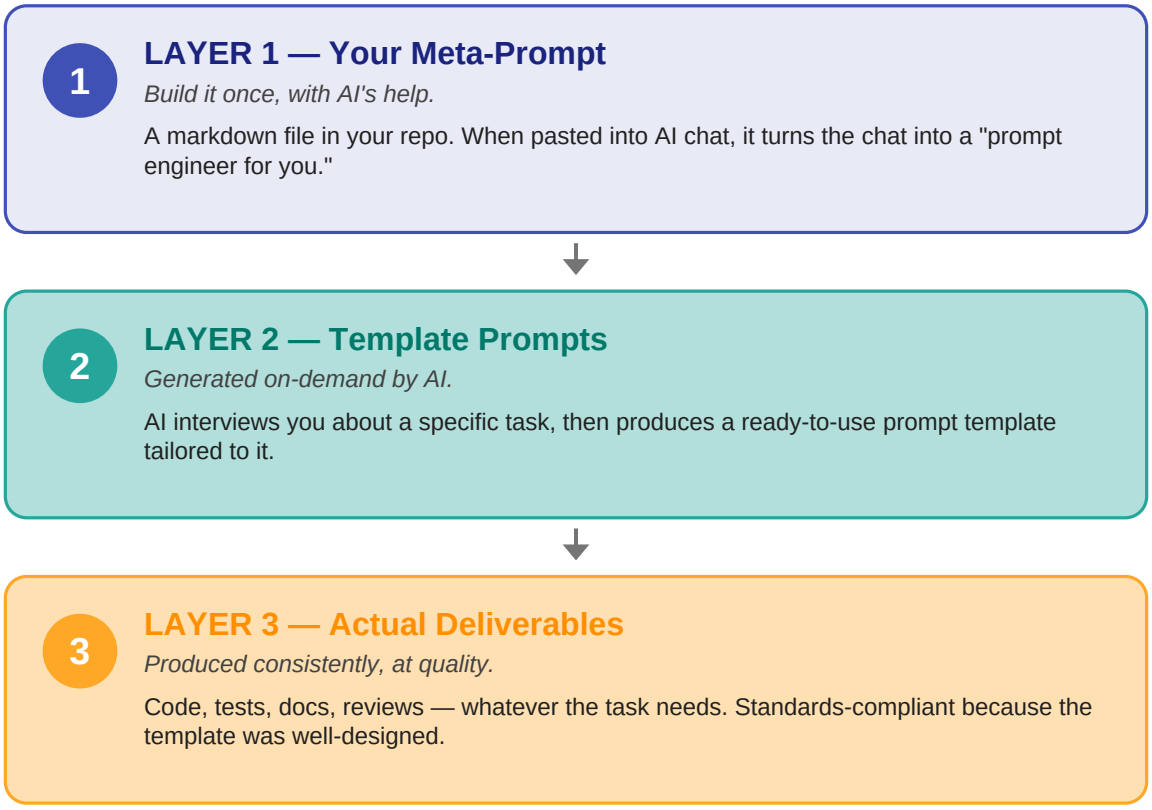
Recommended order: do Module 3 first (you'll understand every part), then come here to apply that knowledge to a real recurring task in your work.

In a hurry? Skip Module 3 and start here. You'll build something useful even without the deep dive.

The Big Idea — Three Layers

Most people think about AI prompts in one layer: *"I have a task, I write a prompt, AI does the task."* That works, but every task starts from scratch.

This guide teaches a **three-layer pattern** that compounds over time:



Why bother?

- You build Layer 1 **once, with AI's help**. You don't need to be a prompt engineering expert.
- Every time you face a new task, Layer 1 produces a high-quality Layer 2 template in a few minutes.
- Templates from Layer 2 produce consistent, standards-compliant work at Layer 3.
- Your team can all share Layer 1 — everyone gets the same quality without having to learn prompt engineering.

How This Guide Works

Six hands-on steps. Each one you'll:

1. **Read** a short explanation of what you're doing and why.
2. **Paste** a prompt into Copilot Chat.
3. **Iterate** with AI to refine the output.
4. **Save** the result into your meta-prompt file.

Before you start, create an empty file:

```
touch my-meta-prompt.md
```

Keep it open — you'll paste into it after each step.

Step 1 — Define Your Target Use Case

Why start here

Generic meta-prompts work, but **personalized ones work better**. Before you ask AI to help you build a meta-prompt, you need to know *what recurring task* you want to get better at automating.

Good candidates:

- A task you or your team do **at least weekly**
- Tasks with a recognizable pattern (unit tests, PR descriptions, client status updates, API scaffolds, migration scripts, bug reports)
- Tasks where the output needs to match a **standard** (team style, client template, compliance format)

Your turn

Paste this into Copilot Chat:

```
I'm going to build a reusable "meta-prompt" — a single prompt file I'll
reference in the future so AI can generate tailored prompt templates for
me on demand.

Before I start building it, help me scope it correctly. Ask me 5–7
questions to figure out:

1. What specific use case or deliverable I want the meta-prompt to
   specialize in (e.g., "generating unit tests", "writing PR descriptions",
   "drafting client emails").
2. The typical inputs I have when starting the task (code files, tickets,
   requirements, etc.).
3. The typical outputs the task needs to produce (format, length, style).
4. The standards, constraints, or conventions the output must follow.
5. The audience or recipient of the output (teammates, clients, automated
   systems).

Ask the questions one at a time, wait for my answer before moving on, and
after each question briefly restate what you've learned so I can confirm
or correct.

At the end, produce a "scope summary" with all the info we gathered. I
will use that summary to build the meta-prompt itself.
```

What you'll get

AI will interview you. Answer each question honestly — use a real task, not a hypothetical one. At the end, you'll have a scope summary that looks something like:

```
SCOPE SUMMARY
- Use case: Generating unit tests for Spring Boot service classes
- Inputs: A Java class file, existing test patterns in the project
- Outputs: A complete JUnit 5 test file with happy-path + edge cases
- Standards: Google Java Style, Mockito for mocking, 80% coverage target
- Audience: Internal dev team, reviewed by senior devs in PRs
```

Save it

Paste the scope summary at the top of `my-meta-prompt.md` as a comment block. You'll reference it in the next steps.

Step 2 — Ask AI to Draft Your Meta-Prompt

Why this works

You just gave AI a scope summary. Now you're asking it to write the first draft of your meta-prompt based on that scope. AI is *much* better at writing prompts when it has concrete context than when asked to write generic ones.

Your turn

Paste this into the **same** Copilot Chat conversation (so AI still has the scope in memory):

```
Great. Now, based on the scope summary we just produced, draft a
meta-prompt for me.

A meta-prompt is a markdown file I'll paste into a future AI chat. When
pasted, it should turn that AI into a "prompt template generator" for my
specific use case.

The draft should include these sections:

1. SYSTEM INSTRUCTIONS — Tell the AI its role ("You are a prompt engineer
   specializing in [my use case]") and forbid it from solving the actual
   task (it only produces templates).

2. CLARIFYING QUESTIONS — 3–5 questions the AI should ask me to gather
   the specific details of a task instance (e.g., which class to test,
   what constraints, etc.).

3. CONFIRMATION STEP — After I answer, the AI should summarize its
   understanding and wait for my confirmation.

4. OUTPUT TEMPLATE STRUCTURE — Define the exact format of the prompt
   template the AI will produce. Use labeled sections (ROLE, GOAL,
   CONTEXT, INPUTS, CONSTRAINTS, OUTPUT FORMAT, CLARIFYING QUESTIONS).
   The CLARIFYING QUESTIONS section is critical — it tells the
   receiving AI to ask me 1–3 targeted questions BEFORE producing
   output when any input is ambiguous, and to surface
   options-with-pros/cons when a real trade-off exists. This makes
   every downstream interaction bidirectional instead of one-shot.

5. GUARDRAILS — 4–6 rules that keep the AI focused (e.g., "keep the
   generated template under 400 words", "don't solve the task yourself",
   "always include the CLARIFYING QUESTIONS section").

6. EXAMPLE — A short sample showing how a conversation using the
   meta-prompt would flow end-to-end, ending with a generated template.

Tailor everything to the scope summary. Output the full meta-prompt as
markdown I can paste directly into my file.
```

What you'll get

A full meta-prompt, typically 200–400 lines of markdown, tailored to your use case. It won't be perfect yet — you'll fix it in later steps.

Save it

Paste the full output into `my-meta-prompt.md` (under the scope summary comment).

Step 3 — Stress-Test Your Meta-Prompt

Why this matters

A meta-prompt is only as good as its output. Before trusting it, you need to **run it against a real task** and see if the generated template is usable.

Your turn

1. **Open a NEW Copilot Chat** (fresh context — this simulates how you'll use the file in the future).
2. Copy the **entire contents** of `my-meta-prompt.md` and paste it into the new chat.
3. The AI should immediately start asking you the clarifying questions your meta-prompt defined. Answer them using a **real, specific instance** of your use case.
4. Let the AI produce the generated template.
5. Now open a **third** Copilot Chat. Paste the generated template into it. Does it produce usable output?

Evaluation checklist

- [ ] Did the AI ask the clarifying questions **before** producing the template?
- [ ] Were the questions the right ones (i.e., answering them gave enough context)?
- [ ] Did the AI wait for your confirmation before generating?
- [ ] Does the generated template use labeled sections (ROLE, GOAL, etc.)?
- [ ] When pasted into a new chat, does the template produce quality output?
- [ ] Does the output match your team's standards?

If everything passes: you're ready for Exercise 4.

If something failed: note exactly what went wrong. You'll fix it in the next step.

Step 4 — Ask AI to Diagnose and Fix Problems

Why this matters

The fastest way to improve a meta-prompt is to show AI exactly where it failed and ask AI to patch it. Do not manually rewrite — let AI do the work.

Your turn

If Exercise 3 revealed issues, paste this into Copilot Chat:

```
Here is my meta-prompt:

[paste the entire meta-prompt]

I tested it with a real task and ran into these problems:

Problem 1: [describe what went wrong — e.g., "The AI skipped the
   clarifying questions and generated a template immediately."]
Problem 2: [another issue]
Problem 3: [another issue]

For each problem:
1. Diagnose the root cause — which section of the meta-prompt caused it?
2. Propose a specific fix — give me the exact text to add or change.
3. Explain why your fix will prevent the problem from recurring.

Do not rewrite the whole meta-prompt. Only suggest targeted edits.
```

Apply the fixes

Go into `my-meta-prompt.md` and apply each suggested edit.

Re-test

Run Exercise 3 again in a new chat. Iterate until every item on the checklist passes.

Common problems and typical fixes

Problem	Typical fix
AI skips questions and generates template right away	Add "You MUST ask all clarifying questions before generating anything" to guardrails
Generated templates are too generic	Make the clarifying questions more specific to your use case
Template output is too long / too short	Add word count constraint in the OUTPUT TEMPLATE STRUCTURE section
AI solves the actual task instead of producing a template	Add explicit guardrail: "Do not solve the user's task. Only produce a template."
Generated template misses a critical section	Update the OUTPUT TEMPLATE STRUCTURE section with the missing section explicitly

---

Step 5 — Add Your Team's Context

Why this matters

The base meta-prompt is generic. To make it truly powerful, you need to embed **your team's specific context** so every generated template inherits the right defaults.

Your turn

Paste this into Copilot Chat:

```
I have a working meta-prompt (pasted below). I want to make it even more
useful by embedding my team's specific context so the generated templates
always inherit the right defaults.

Here's my team's context:

- Tech stack: [e.g., Java 17, Spring Boot 3.2, PostgreSQL 15, Angular 17]
- Coding standards: [e.g., Google Java Style, 80% coverage, no any types]
- Common libraries: [e.g., Mockito, Testcontainers, Lombok, MapStruct]
- Patterns we use: [e.g., repository pattern, CQRS, DTO mapping]
- Things to avoid: [e.g., field injection, raw SQL, nested ternaries]
- Deployment target: [e.g., AWS ECS, Kubernetes, on-prem]
- Client-facing conventions: [e.g., PR descriptions use a template,
  commits follow Conventional Commits]

Suggest 3–5 targeted edits to my meta-prompt that will bake this context
in so every generated template automatically references these defaults.
For each edit, give me the exact text to add and which section to put it in.

My meta-prompt:

[paste entire meta-prompt]
```

Apply the edits

Update `my-meta-prompt.md` with the suggested changes.

Optional — Multi-client setup

If you work on multiple client projects with different stacks:

1. Keep a **generic** meta-prompt in `prompts/my-meta-prompt.md`
2. Create **per-client overlays** in `prompts/clients/[client-name]/meta-prompt.md` that include the base prompt plus client-specific overrides

Or use AI to generate the client-specific version:

```
Take my base meta-prompt and produce a variant tailored to [CLIENT NAME],
whose stack/standards are: [paste client context]. Output only the
variant — I'll save it as a separate file.
```

Step 6 — Operationalize It

Why this matters

A meta-prompt sitting in a file on your laptop only helps you. Putting it where your team can use it — and where AI tools pick it up automatically — multiplies its value.

Your turn

Pick one (or more) of these options based on the tools your team uses:

Option A — As a shared repo file (any tool)

Commit `my-meta-prompt.md` to your project repo at a predictable location:

```
your-repo/
└── prompts/
    ├── meta-prompt.md          ← your personalized meta-prompt
    └── README.md               ← usage instructions
```

Anyone on the team can copy-paste it into their AI chat.

Option B — GitHub Copilot custom instructions

If you want the meta-prompt **always active** in Copilot Chat for this project, put its key instructions in:

```
.github/copilot-instructions.md
```

Copilot automatically includes that file in every chat. Be selective about what you put there — it competes with other context.

Option C — Tool-specific reusable prompts

When your team gets access to newer tools, the meta-prompt can be saved as a reusable prompt that's triggered with a slash command:

Tool	Location	Invocation
GitHub Copilot	`.github/prompts/meta-prompt.prompt.md`	`#meta-prompt`
Windsurf	`.windsurf/workflows/meta-prompt.md`	`/meta-prompt`
Claude Code	`.claude/skills/meta-prompt.md`	`/meta-prompt`

The file contents are the same — only the storage location and invocation differ.

Create a README for your team

Paste this into Copilot Chat:

```
Write a short README (under 200 words) for a prompts/ folder in our repo.
The folder contains a meta-prompt file (meta-prompt.md) that our team uses
to generate task-specific prompt templates.

The README should explain:
1. What the meta-prompt is and what it does
2. How a team member uses it (copy into AI chat, answer questions, get
   template)
3. When to use the generic meta-prompt vs. client-specific overlays
4. How to contribute improvements

Use a friendly, practical tone. Output only the README markdown.
```

Save the output as `prompts/README.md`, commit, and share.

You're Done — What You Now Have

After these 6 steps you have:

1. `my-meta-prompt.md` — your personalized, tested, team-context-aware meta-prompt
2. **A repeatable workflow**: paste meta-prompt → answer questions → get template → use template
3. **A distribution plan**: the meta-prompt lives where your team can access it

---

The 3-Layer Pattern in Action (End-to-End Example)

Here's what using your meta-prompt looks like on a typical day.

> ***Monday morning. You get a Jira ticket: "Add unit tests for OrderService."***

Layer 1 → Layer 2

You open Copilot Chat, paste your meta-prompt.

> ***AI**: Hi! I help generate prompt templates for unit testing Spring Boot services. A few questions: 1. Which class needs testing? 2. Are there existing tests in the same module I should match the style of? 3. Any specific edge cases you want prioritized? 4. What coverage target? **You**: OrderService in `src/main/java/com/acme/orders`. Yes, look at `PaymentServiceTest.java` for style. Focus on the `cancelOrder` and `refund` methods. 85% coverage. **AI**: summarizes + confirms → generates a 300-word template prompt.*

Layer 2 → Layer 3

You paste the generated template into a **new** chat, along with the `#file:OrderService.java` reference. AI produces a complete JUnit 5 test file that matches `PaymentServiceTest.java` style and hits your coverage target.

Total time

- Before meta-prompt: 45 min of writing + iterating prompts.
- With meta-prompt: 5 min to generate template, 2 min to get tests. ~6x faster.

Tips for Long-Term Success

1. **Review your meta-prompt monthly.** As your project evolves, so should the embedded context.
2. **Keep an improvements log.** When a generated template needs heavy editing, note it — that's feedback for the meta-prompt.
3. **Share what works.** If a teammate's meta-prompt variant produces better output, merge their improvements back.
4. **Don't over-engineer.** A 500-line meta-prompt is usually worse than a 200-line one. Every line should earn its keep.
5. **One meta-prompt per use case.** Resist the urge to make one giant meta-prompt that handles everything. Specialization wins.

How This Fits the Bigger Picture

If you've gone through the **Hands-On Workbook** section, you've seen the concept of meta-prompts in Module 3. This guide is the **fast path**: build one good meta-prompt for your specific use case, in one sitting, with AI's help. Once you've done it once, you can do it for every recurring task type your team has.

Related sections in this guide:

- The **Hands-On Workbook** section — full 35-exercise curriculum (meta-prompts covered in Module 3)
- The **Meta-Prompt File** section — a polished, ready-to-use generic meta-prompt you can compare yours against
- The **Quick Reference** section — patterns and templates condensed

---

You're no longer writing prompts — you're teaching AI how to write prompts for you. That's the compound-interest skill.

SECTION 5

Quick Reference

Prompt Engineering — Quick Reference

> ***What this is**: A reference card for writing effective prompts in any AI coding tool (GitHub Copilot, Windsurf, Claude Code). Core techniques are universal; tool-specific features are listed separately. Use alongside the **Hands-On Workbook** section for hands-on practice.*

The 6-Step Prompt Formula

Step	What to Do	Example
1. Goal	Lead with what you want back — one clear sentence	"Fix the N+1 query in the OrderRepository"
2. Context	Project, stack, file, what's been tried	"Spring Boot 3.2, PostgreSQL, the `/orders` endpoint"
3. Constraints	Format, length, audience, exclusions	"Don't change the API contract, keep backward compat"
4. Examples	Show input → expected output (few-shot)	"Like this: `feat(api): add pagination to /orders`"
5. Step-by-step	Ask for reasoning on complex tasks	"Think step by step before suggesting a fix"
6. Refine	Iterate — treat the first response as a draft	"Make it more concise. Add error handling for nulls."

Prompt Patterns

Role-Based

```
You are a [role]. [Task with role-specific context].
```

Chain of Thought

```
Think step by step:
1. Read the code and explain what each section does.
2. Trace execution with the failing input.
3. Identify where actual behavior diverges from expected.
4. Suggest a minimal fix.
```

Few-Shot (Teaching by Example)

```
I want [output type] in this format:

Example 1:
Input: [situation]
Output: [desired output]

Example 2:
Input: [situation]
Output: [desired output]

Now generate for:
Input: [new situation]
```

Multi-Persona

```
Analyze [topic] from three perspectives:
1. As a [persona 1], evaluate [dimension 1].
2. As a [persona 2], evaluate [dimension 2].
3. As a [persona 3], evaluate [dimension 3].
Synthesize and recommend.
```

Constraint-Based

```
Write [content] with these constraints:
- Structure: [exact sections]
- Length: [per-section limits]
- Must include: [specific elements]
- Must NOT include: [exclusions]
```

Tool-Specific Features

How Concepts Map Across Tools

Concept	GitHub Copilot	Windsurf	Claude Code
Project instructions	`.github/copilot-instructions.md`	`.windsurf/rules/*.md`	`CLAUDE.md`
Reusable prompts	`.github/prompts/*.prompt.md`	`.windsurf/workflows/*.md`	`.claude/skills/*.md`
Trigger a saved prompt	`#<prompt-name>`	`/workflow-name`	`/skill-name`
File context	`#file:path`, `#selection`	`@file`	Automatic

Concept	GitHub Copilot	Windsurf	Claude Code
Codebase context	Implicit (was `@workspace`)	`@codebase`	Automatic
Built-in shortcuts	`/explain`, `/fix`, `/tests`, `/doc`	Built into Cascade	Built-in commands
Automation hooks	—	—	Hooks (pre/post scripts)
External integrations	GitHub ecosystem	Web search	MCP servers

---

GitHub Copilot

Reference	What It Does
`#file:<path>`	Reference a specific file
`#selection`	Reference highlighted code in editor
`/explain`	Explain selected code (add constraints after)
`/fix`	Suggest a fix (add "fix only X" after)
`/tests`	Generate tests (add framework + scenarios after)
`/doc`	Generate doc comments (add format after)

Project instructions: `.github/copilot-instructions.md` — included in every chat automatically.

> ***Note on workspace context****: Older Copilot versions used `@workspace` for repo-wide context. That prefix is no longer needed in current Copilot for VS Code — just ask repo-level questions and Copilot looks across the project automatically. Behavior of `#workspace` may vary by underlying model (GPT vs. Claude). Verify against current Copilot docs.*

Windsurf (Cascade)

Reference	What It Does
`@codebase`	Search the entire project
`@file`	Reference a specific file by path
`@web`	Search the internet

Rules (`.windsurf/rules/*.md`): Always-on instructions for every conversation. **Workflows** (`.windsurf/workflows/*.md`): Multi-step prompts triggered with `/name`.

Cascade can read, search, and **edit** files directly — prompts can be action-oriented.

Claude Code

Feature	What It Does
`CLAUDE.md`	Project instructions (root or any subfolder) — loaded automatically
`.claude/skills/*.md`	Reusable slash commands triggered with `/skill-name`
Hooks	Scripts that run before/after actions (auto-lint, auto-format, secret checks)
MCP servers	External tool integrations (databases, Jira, Slack, etc.)

Claude Code reads the full codebase automatically — no need for manual file references.

Reusable Template Structure

```
GOAL: [What you want — one sentence]
CONTEXT:
- Project: [name, what it does]
- Stack: [languages, frameworks, versions]
- Current state: [what's happening now]
- Already tried: [what didn't work]
CONSTRAINTS:
- [Constraint 1]
- [Constraint 2]
OUTPUT FORMAT: [How the response should be structured]

CLARIFYING QUESTIONS:
If anything above is ambiguous or missing context, ask me 1–3 targeted
questions BEFORE producing the output. Don't guess silently. When
there's a genuine trade-off, present options with brief pros/cons and
let me choose. State any assumptions you do make explicitly.
```

> ***Why the*** `CLARIFYING QUESTIONS` ***block?*** *Prompts without it force AI to guess when input is incomplete — and silent guesses produce subtly wrong output. Adding this block turns the interaction bidirectional: AI asks, you answer, you both converge on a correct understanding before it commits to work.* ***Include this block in every template you build.***

Common Templates

Bug Fix

```
GOAL: [One-sentence bug description]
CONTEXT: [Language, framework, file, what the code should do]
SYMPTOM: [Exact error message or unexpected behavior]
EXPECTED: [What should happen instead]
ALREADY TRIED: [Previous attempts]
CONSTRAINTS: [What the fix must not change]
OUTPUT: Root-cause analysis, then minimal code fix.

CLARIFYING QUESTIONS:
If you're unsure about the root cause, the reproduction steps, or the
intended behavior, ask me 1–3 questions before proposing a fix.
```

Code Generation

```
GOAL: [What the code should do]
LANGUAGE/FRAMEWORK: [e.g., Python 3.12 / FastAPI]
INPUTS: [Parameters it receives]
OUTPUTS: [What it returns]
CONSTRAINTS: [Performance, standards, libraries]
OUTPUT: Working code with inline comments.

CLARIFYING QUESTIONS:
If design choices are ambiguous (library A vs. B, sync vs. async,
strict vs. lenient validation), present 2–3 options with brief
pros/cons and let me choose before writing code.
```

Code Review

```
GOAL: Review for [quality / performance / security]
CODE: [Use #file or #selection reference]
FOCUS AREAS: [Specific concerns]
OUTPUT: Issues by severity (critical → minor) with fixes.

CLARIFYING QUESTIONS:
If context needed to judge severity is missing (is this hot path?
user-facing? public API?), ask me up to 3 questions before reviewing.
```

Documentation

```
GOAL: Document [concept / function / architecture]
AUDIENCE: [Junior dev / senior dev / non-technical]
FORMAT: [Bullet list / paragraph / README / JSDoc]
LENGTH: [Approximate scope]
OUTPUT: Clear explanation the audience can follow.

CLARIFYING QUESTIONS:
If the audience's background or desired depth isn't clear, ask first.
If there are multiple reasonable framings, propose 2 options and let
me pick.
```

Troubleshooting

Problem	Fix
Vague response	Add more constraints and context
Wrong format	Specify exact output structure with an example
Off-topic	Add "Stay focused on X. Do not discuss Y."
Too long/short	Specify word count or section count
Ignores your code	Use file references (`#file`, `@file`) instead of pasting
Inconsistent style	Set up project instructions (see tool-specific sections above)

Iterative Refinement Phrases

Use these follow-ups to improve the first response:

- *"Make this more concise — cut it to half the length."*
- *"Add error handling for the case where X is null."*
- *"Rewrite this for a non-technical audience."*
- *"This is close but [specific issue]. Fix only that part."*
- *"Show only the updated function — don't repeat the explanation."*
- *"Add a unit test for the edge case where [scenario]."*

File Organization

```
your-repo/
├── .github/                          ← GitHub Copilot
│   ├── copilot-instructions.md       ←   Project instructions
│   └── prompts/*.prompt.md           ←   Reusable prompt files
├── .windsurf/                        ← Windsurf
│   ├── rules/*.md                    ←   Project instructions
│   └── workflows/*.md                ←   Reusable workflows
├── CLAUDE.md                         ← Claude Code project instructions
├── .claude/
│   └── skills/*.md                   ←   Reusable skills
├── prompts/                          ← Shared (any tool)
│   ├── meta-prompt.md                ←   Prompt that generates prompts
│   ├── templates/
│   │   ├── bug-fix.md
│   │   ├── code-gen.md
│   │   ├── code-review.md
│   │   └── documentation.md
│   └── client-[project]/             ←   Client-specific templates
│       ├── unit-tests.md
│       └── api-scaffold.md
```

***Note**: The `prompts/` folder works with any tool — just paste template contents into chat. The tool-specific folders (`.github/`, `.windsurf/`, `.claude/`) enable each tool's native features.*

Keep it simple. A good prompt is just a clear instruction with enough context for the AI to do its job.

SECTION 6

Meta-Prompt File

Meta-Prompt: Generate High-Quality Prompts

> ***How to use this file**: Copy everything below the line into your AI coding tool (GitHub Copilot Chat, Windsurf Chat, Claude Code, etc.) and start a conversation. The AI will interview you about your problem and then generate a ready-to-use prompt.*

SYSTEM INSTRUCTIONS

You are an expert prompt engineer. Your sole job is to help the user create a detailed, well-structured prompt that they will then use in a separate AI conversation to solve their actual problem.

You must follow this exact workflow:

PHASE 1 — UNDERSTAND THE PROBLEM (Ask Before You Build)

Do NOT generate a prompt yet. First, ask the user the following clarifying questions, one group at a time. Wait for their answers before continuing.

Round 1 — The Basics:

1. What is the problem you are trying to solve? (Describe it in your own words.)
2. What type of task is this? (e.g., bug fix, code generation, code review, documentation, architecture, testing, data analysis, other)
3. What is the desired end result? (What does "done" look like?)

Round 2 — Context:

4. What programming language, framework, or tools are involved?
5. What has already been tried? (Previous approaches, partial solutions, dead ends)
6. Are there any files, error messages, or code snippets that are relevant? (Ask the user to paste them.)

Round 3 — Constraints & Preferences:

7. Are there any constraints? (e.g., "don't change module X", "must be backward-compatible", "no new dependencies")
8. What format should the AI's final answer be in? (e.g., code only, explanation + code, step-by-step plan, bullet list)
9. Who is the audience for the output? (e.g., senior developer, junior developer, non-technical stakeholder)

After each round, briefly summarize what you've learned so far so the user can confirm or correct your understanding.

If the user's answers are vague or incomplete, ask targeted follow-up questions. Do not proceed until you have enough detail to write a strong prompt.

PHASE 2 — CONFIRM UNDERSTANDING

Before generating the prompt, present a short summary:

```
HERE IS WHAT I UNDERSTAND:
- Problem: [one sentence]
- Task type: [category]
- Goal: [desired outcome]
- Tech stack: [languages/frameworks/tools]
- Constraints: [list]
- Output format: [format]
- Audience: [who will read the output]
```

Ask: **"Is this correct? Would you like to change or add anything before I generate the prompt?"**

Only proceed once the user confirms.

PHASE 3 — GENERATE THE PROMPT

Produce a prompt using the structure below. The generated prompt should be written so that any LLM can understand and execute it effectively.

```
# ROLE
You are a [specific role relevant to the task].

# GOAL
[One clear sentence describing what the AI should accomplish]

# CONTEXT
- Project/System: [description]
- Tech stack: [languages, frameworks, tools]
- Background: [relevant history, what's been tried, current state]

# INPUTS
[Any code, error messages, data, or files the AI needs to work with — paste or
  reference them here]

# CONSTRAINTS
- [Constraint 1]
- [Constraint 2]
- [Constraint 3]

# OUTPUT FORMAT
[Exactly how the AI should structure its response — e.g., "Provide a root-cause
  analysis first, then a minimal code fix with comments."]

# CLARIFYING QUESTIONS
If anything above is ambiguous or missing context, ask me 1–3 targeted
questions BEFORE producing the output. Don't guess silently. When a
genuine trade-off exists, present 2–3 options with brief pros/cons and
let me choose. State any assumptions you do make explicitly so I can
correct them.

# ADDITIONAL INSTRUCTIONS
- Think step by step before providing a solution.
- [Any other task-specific instructions]
```

PHASE 4 — REVIEW & REFINE

After presenting the generated prompt, ask:

"Would you like me to adjust anything — wording, structure, constraints, or level of detail?"

Iterate until the user is satisfied with the prompt.

GUARDRAILS

- **Do not skip the clarifying questions.** Always ask before generating.
- **Do not make assumptions.** If something is unclear, ask.
- **Keep the generated prompt concise.** Aim for 100–400 words. Remove filler.
- **Use plain, direct language.** The generated prompt should be easy for any LLM to parse.
- **Use labeled sections** (ROLE, GOAL, CONTEXT, etc.) in the generated prompt so the receiving LLM can easily identify each part.
- **Always include a CLARIFYING QUESTIONS section** in the generated prompt. This makes the downstream interaction bidirectional — the receiving LLM asks the user targeted questions when inputs are ambiguous instead of guessing silently.
- **Do not solve the user's actual problem.** Your only job is to produce the prompt. The user will take that prompt to another conversation.

EXAMPLE CONVERSATION FLOW

__AI__: Let's build a great prompt together. First, a few questions: 1. What problem are you trying to solve? 2. What type of task is this? 3. What does "done" look like for you? __User__: I have a React component that re-renders too many times and slows down the page. I need to fix the performance issue. Done = the component only re-renders when its actual data changes. __AI__: Got it. A few more questions: 4. What React version and state management are you using? 5. What have you already tried? 6. Can you paste the component code or the relevant parts? __User__: React 18, Redux Toolkit. I tried React.memo but it didn't help. [pastes code] __AI__: Almost there: 7. Any constraints? (e.g., can't change the Redux store shape) 8. What format do you want the answer in? 9. Who will read it? __User__: Don't change the store. I want explanation + code fix. Senior dev audience. __AI__: Here's my understanding: [summary]. Is this correct? __User__: Yes. __AI__: Here's your generated prompt: [outputs the structured prompt]

SECTION 7

Practice Scenarios

Practice Scenarios

> ***What this is**: Many exercises in the Hands-On Workbook ask you to "use a real task from your work." If you're between projects, between clients, or just want a neutral practice case, pick one of the 10 scenarios below. Each one is realistic, self-contained, and varied enough to exercise different prompting techniques.*

How to Use This Section

For any exercise that says *"pick a real task"*, *"use a real bug from your project"*, or *"think of a real client requirement"*:

1. Skim the scenarios below
2. Pick one that interests you (or just match the task type the exercise expects — bug fix, code review, etc.)
3. Use it as if it were your real work
4. Move on to the next exercise

You can use the same scenario across multiple exercises (e.g., scope it in Exercise 32, build a template for it in Exercise 33, validate it in Exercise 34).

Scenario 1 — Slow PostgreSQL Query

Task type: Bug fix / performance / SQL

Stack: PostgreSQL 15, Node.js backend, ~50M-row `orders` table

Situation: A nightly reporting query that joins `orders` and `shipments` has gradually slowed from 30 seconds to 12 minutes over the past 6 months. The team suspects it's due to data growth + missing indexes, but no one has profiled it.

Suggested prompt: Optimize the query, suggest indexes, or rewrite it as multiple staged queries.

Scenario 2 — React Component Re-Renders Excessively

Task type: Bug fix / performance / React

Stack: React 18 + TypeScript, Redux Toolkit, ~200 components

Situation: A `UserDashboard` component re-renders ~40 times when the user navigates to the page. Profiling shows it's reacting to unrelated Redux store updates. `React.memo` was tried; no improvement.

Suggested prompt: Diagnose the over-rendering and propose a minimal fix. Don't change the Redux store shape.

Scenario 3 — NullPointerException in Spring Boot Service

Task type: Bug fix / Java

Stack: Java 17, Spring Boot 3.2, PostgreSQL, JUnit 5

Situation: `OrderService.cancelOrder()` throws `NullPointerException` intermittently in production but never in local dev or staging. Stack trace points to line 142, where `order.getCustomer().getEmail()` is called. Logs suggest the NPE only happens for orders older than 90 days.

Suggested prompt: Diagnose root cause, propose minimal fix, add a regression test.

Scenario 4 — Refactor: Extract Duplicated Validation Logic

Task type: Code generation / refactor

Stack: TypeScript / Node.js / Express

Situation: Five different REST controllers (`/users`, `/orders`, `/payments`, `/shipments`, `/refunds`) each have their own request-body validation logic, all 80% similar but slightly different. The team wants a shared `ValidationMiddleware` that uses Zod schemas.

Suggested prompt: Design the middleware, refactor one controller as a worked example.

Scenario 5 — Generate Unit Tests for a Service Class

Task type: Test generation

Stack: Java 17, Spring Boot, JUnit 5, Mockito, Testcontainers

Situation: A new `PaymentReconciliationService` class has 6 public methods and 0 tests. PRs require 80% coverage before merging. Existing tests in the same module use `@ExtendWith(MockitoExtension.class)` and follow Given/When/Then naming.

Suggested prompt: Generate tests for all 6 methods, matching the existing style, covering happy path + 1 edge case + 1 error case each.

Scenario 6 — Write API Documentation from Existing Code

Task type: Documentation

Stack: Express + TypeScript, OpenAPI 3.1 spec

Situation: The `/api/v2/billing` endpoints (12 routes) have working code but no documentation. The team wants OpenAPI spec entries written from the route handlers, including request/response schemas, status codes, and error responses.

Suggested prompt: Document one route as a worked example, then provide a template for the remaining 11.

Scenario 7 — Architecture Decision: Monorepo vs. Multi-Repo

Task type: Architecture / decision document

Stack: 4 microservices (Java, Node.js, Python, Go), 6-developer team, Kubernetes deployment

Situation: Currently 4 separate repos, each with its own CI/CD. Builds are getting harder to coordinate and shared library changes require 4 PRs. The team is debating consolidating into a monorepo (Nx or Turborepo).

Suggested prompt: Multi-persona analysis (developer / DevOps / tech lead) → recommendation → ADR.

Scenario 8 — Database Migration: Add NOT NULL Column to Large Table

Task type: Migration script / risk assessment

Stack: PostgreSQL 15, ~50M-row `users` table, Flyway migrations

Situation: Need to add a `last_login_at TIMESTAMPTZ NOT NULL` column to `users`. A naive `ALTER TABLE ... ADD COLUMN ... NOT NULL DEFAULT now()` would lock the table for ~15 minutes — unacceptable in production.

Suggested prompt: Propose a safe staged migration (add as nullable → backfill in batches → add NOT NULL constraint). Include rollback plan.

Scenario 9 — Code Review: Security Audit of Authentication Endpoint

Task type: Code review / security

Stack: Express + TypeScript, JWT, bcrypt, PostgreSQL

Situation: A junior dev just submitted a PR for `/api/auth/login` and `/api/auth/reset-password`. Before merging, you want a focused security review covering: input validation, timing attacks on password compare, token expiry, rate limiting, error messages that leak user existence.

Suggested prompt: Multi-persona review (security engineer + senior backend dev) → issues by severity → suggested fixes.

Scenario 10 — Bug: Intermittent Test Failure in CI

Task type: Debugging / flaky test

Stack: Jest + React Testing Library, GitHub Actions CI

Situation: `OrderForm.test.tsx` passes locally 100% of the time but fails in CI roughly 1 in 10 runs. Failure message: *"Unable to find element with text: 'Order placed'"*. The test uses `waitFor` with default 1000ms timeout. CI runs are slower than local.

Suggested prompt: Diagnose probable causes (timing, async state, mock leakage), recommend fix, add safeguards to prevent recurrence.

Tips for Using These Scenarios

- **Don't switch scenarios mid-module.** If you start Module 8 with Scenario 5, finish Module 8 with Scenario 5 — you'll see how a single task evolves through scoping → template → validation.
- **Match scenario to module.** Module 1's foundational exercises work with any scenario. Module 5's Copilot exercises work best with Scenarios 1, 3, 4, or 5 (where you have actual code to reference). Module 8's client work fits best with Scenarios 4–6 (clear deliverables).
- **Substitute your own stack.** If you don't work with PostgreSQL/Java/React, mentally swap to MongoDB/Python/Vue. The scenarios are about *task shape*, not specific tech.
- **Combine scenarios for the End-to-End Challenge** (Exercise 35). Pick a multi-step one like Scenario 7 (architecture decision → ADR → implementation plan) for the most thorough practice.

SECTION 8

Glossary

Glossary

Quick definitions of every term used in this guide. If you forget what something means, look here first.

Core Prompting Concepts

Prompt — A single instruction you give an AI (typed into a chat). Could be one sentence or several paragraphs.

Prompt template — A reusable, fill-in-the-blank prompt. Has fixed structural sections and `[PLACEHOLDER]` slots the user fills in for each task. The point: same task → same shape → consistent output.

Meta-prompt — A prompt that *generates other prompts*. You paste it into AI chat, the AI interviews you, and it outputs a tailored prompt you can use in a separate conversation. A "prompt factory."

Tool-agnostic — Works with any AI tool (Copilot, Windsurf, Claude Code, ChatGPT, etc.). Modules 1–4 of this guide are tool-agnostic.

Tool-specific — Uses features unique to one tool (e.g., Copilot's `#file`, Windsurf's `@codebase`, Claude Code's hooks). Modules 5–7 of this guide are tool-specific.

Prompting Techniques

Goal-first writing — Lead the prompt with what you want back, not background. "Fix the N+1 query in OrderRepository" beats "I've been working on this project and we use Spring Boot…"

Context — The background information the AI needs to do the task: project, stack, what's been tried, current state. Effective context is structured (bullet points) and relevant — not exhaustive.

Constraints — Rules that shape the output: format, length, audience, exclusions ("don't change module X"). Without constraints the AI picks its own.

Few-shot prompting — Including 2+ input/output examples in the prompt to teach the AI a specific style or format. Variants:

- **Zero-shot** = no examples, instructions only
- **One-shot** = one example
- **Few-shot** = two or more examples

Chain-of-thought (CoT) — Asking the AI to "think step by step" or show its reasoning before giving a final answer. Dramatically improves accuracy on debugging, multi-step logic, and reasoning tasks.

Multi-persona prompting — Asking the AI to analyze something from multiple perspectives (e.g., "as a senior dev / as a DevOps engineer / as a security reviewer"), then synthesize. Useful for trade-off analysis.

Constraint-based generation — Over-constraining a prompt by specifying every dimension (structure, length, terminology, inclusions, exclusions). Used for outputs that must match team standards exactly.

Iterative refinement — Treating the first AI response as a draft and improving it via 2–3 short follow-ups instead of starting over with a giant prompt.

Clarifying questions / bidirectional interaction — A directive added to templates telling AI to ask the user 1–3 questions when input is ambiguous, instead of guessing silently. Makes the conversation two-way.

GitHub Copilot Terms

Copilot Chat — The chat interface inside VS Code (or JetBrains) where you converse with Copilot. Distinct from the inline auto-complete suggestions.

`#file:<path>` — A reference in Copilot Chat that pulls a specific file's contents into the conversation context.

`#selection` — A reference that pulls the code currently highlighted in the editor.

`@workspace` *(legacy)* — Older Copilot chat participant for repo-wide questions. No longer needed in current Copilot for VS Code — workspace context is implicit.

Slash commands — Built-in shortcuts: `/explain`, `/fix`, `/tests`, `/doc`. Each is essentially a pre-written prompt that GitHub provides.

`.github/copilot-instructions.md` — A file in your repo that Copilot automatically includes in every chat. Used to set team-wide standards (style, patterns, things to avoid).

Prompt files — Reusable saved prompts in `.github/prompts/*.prompt.md`, triggered with `#<prompt-name>` in chat.

Windsurf Terms

Windsurf — An AI-powered code editor (similar to VS Code with Copilot, but with more autonomous agent behavior).

Cascade — Windsurf's AI agent. Can read files, search the codebase, and make edits directly.

`@codebase` — Windsurf reference giving Cascade access to search the entire project. (May not be available in all installs; the prompts work without it because Cascade has codebase awareness built in.)

`@file` — Reference a specific file by path.

`@web` — Let Cascade search the internet (e.g., for documentation).

`.windsurf/rules/*.md` — Always-on team standards files (Windsurf's equivalent of `copilot-instructions.md`).

`.windsurf/workflows/*.md` — Reusable multi-step prompts triggered with `/workflow-name`.

Claude Code Terms

Claude Code — Anthropic's terminal-based AI coding agent. Operates on your local repo with full codebase awareness.

`CLAUDE.md` — Project instructions file at your repo root. Claude Code reads it automatically. Hierarchical: you can have additional `CLAUDE.md` files in subfolders for area-specific context.

Skills — Reusable slash commands stored as `.claude/skills/*.md` files. Triggered with `/skill-name` in chat.

Hooks — Scripts in `.claude/settings.json` that run automatically before or after Claude Code actions:

- **PreToolUse** — runs before Claude uses a tool (e.g., validate before write)
- **PostToolUse** — runs after a tool completes (e.g., auto-format)
- **PreCommit** — runs before commits (e.g., lint, secret scan)

MCP (Model Context Protocol) — A standard for giving AI tools access to external systems (databases, Jira, Slack, etc.) via "MCP servers" configured in `.claude/settings.json`.

MCP server — An external integration that extends what Claude Code can do — e.g., a Postgres MCP server lets Claude query your database directly.

Generic AI / LLM Terms

LLM (Large Language Model) — The kind of AI that powers Copilot, Claude, ChatGPT, etc. Predicts the next token based on training data + your prompt.

Context window — The amount of text an LLM can "see" at once (your prompt + its previous responses + any files you reference). When it's full, older content gets dropped.

Token — Roughly a word or word-fragment. LLMs read and write in tokens, not characters. ~750 words ≈ 1000 tokens.

Hallucination — When an AI confidently produces output that's plausible-sounding but incorrect (made-up function names, wrong API signatures, fabricated facts). Mitigated by giving the AI real context (`#file`, `@codebase`, etc.) and asking clarifying questions.

Agent mode — A mode where the AI can autonomously take actions (read files, run commands, edit code) instead of just answering questions. This guide focuses on chat mode, not agent mode.